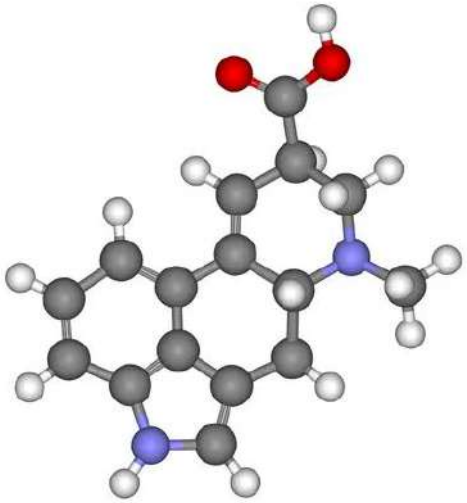
Sur le problème d'isomorphisme de groupes

JEON Saehyun Davin n° 29566

Partenaire: BOURILLON Jules n°44227

Introduction

Cryptographie



Chimie moléculaire

Cadre de résolution

Enoncé du problème

IsoGroupe

Entrée : $n \in \mathbb{N}^*$, deux groupes G_1 et G_2 (qu'on identifie à $[\![0, n-1]\!]$) tels que $|G_1| = |G_2| = n$, donnés par leurs tables de Cayley.

Sortie :

$$\begin{cases} \text{Non} & \text{si } G_1 \not\simeq G_2, \\ (\text{Oui}, \phi) & \text{où } \phi : G_1 \to G_2 \text{ est un isomorphisme explicite} \end{cases}$$

Motivation:

- **Accès en O(1) à l'ordre du groupe**
- **Calculs en O(1)**
- **Facile de coder soit même des exemples de groupes**

Plan

I) Implémentation pour le cas général: l'algorithme de Tarjan

II) Le cas particulier des groupes abéliens: l'algorithme de Nader H. Bshouty

III) Analyse des résultats

Transition

I - Implémentation pour le cas général: l'algorithme de Tarjan

I - Implémentation pour le cas général

Un algorithme naïf ?

NAÏF

Entrée : Deux groupes G et G'.

Sortie : VRAI si $G \cong G'$, FAUX sinon.

Pour chaque bijection $f : G \to G'$:
  Si $\forall (x, y) \in G \times G, f(x \cdot y) = f(x) \cdot f(y)$:
    Renvoyer VRAI
Renvoyer FAUX

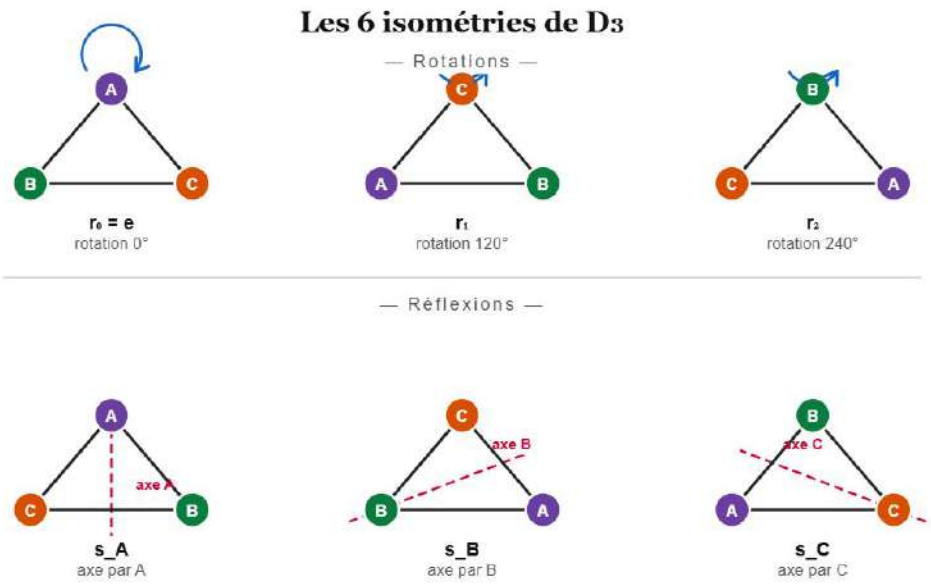
Complexité: $\Omega(n^2 n!)$ (même en haut niveau)...

Solution ? Utiliser des générateurs.

I - Implémentation pour le cas général

Etude d'un cas concret

Le groupe D3 des isométries du triangle

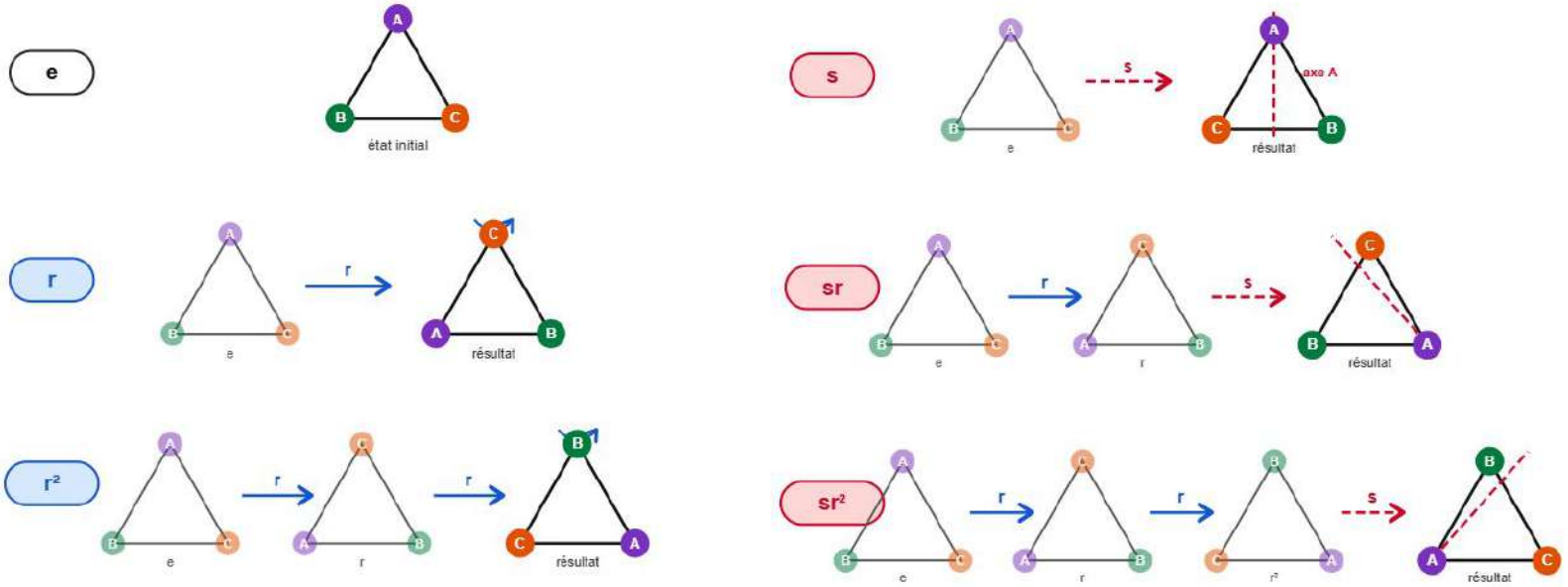


Le groupe S3, des permutations de {1, 2 3}

Permutation	Tableau	Effet sur [1,2,3]
()	[1,2,3]	[1,2,3]
(1 2 3)	[1,2,3]	[2,3,1]
(1 3 2)	[1,2,3]	[3,1,2]
(1 2)	[1,2,3]	[2,1,3]
(2 3)	[1,2,3]	[1,3,2]
(1 3)	[1,2,3]	[3,2,1]

I - Implémentation pour le cas général

Etude d'un cas concret



D3 est généré par deux éléments:

$r = r^{\circ}120$ et $s = sA$

I - Implémentation pour le cas général

Etude d'un cas concret

Décomposition	Permutation equivalente	Effet sur [1,2,3]
Id	()	[1,2,3]
r'	(1 2 3)	[2,3,1]
r'*r'	(1 3 2)	[3,1,2]
s'	(1 2)	[2,1,3]
s'*r'	(1 3)	[3,2,1]
s'*r'*r'	(2 3)	[1,3,2]

S3 est généré par deux éléments: $r' = (1\,2\,3)$ et $s' = (1\,2)$

I - Implémentation pour le cas général

Etude d'un cas concret

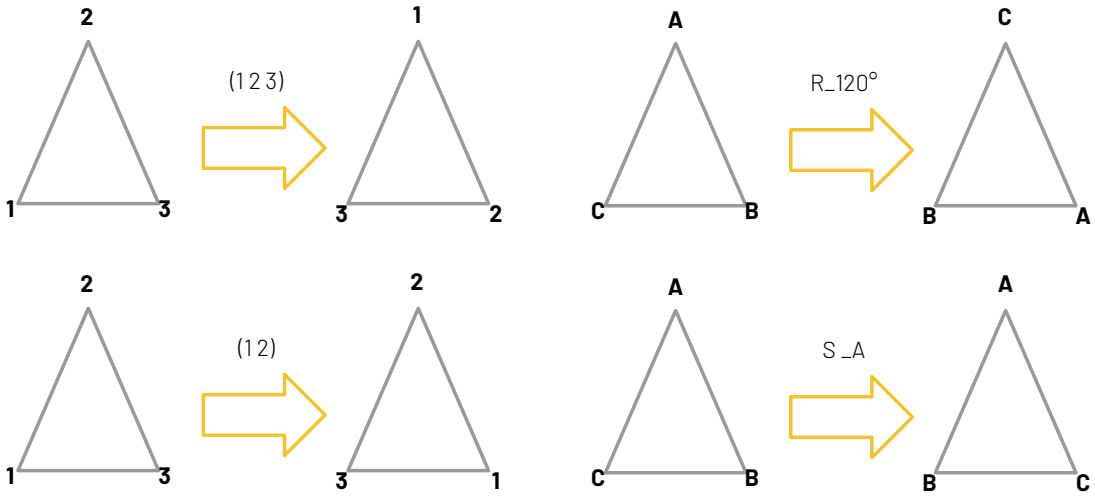
Décomposition	Permutation equivalente	Effet sur [1,2,3]
Id	()	[1,2,3]
r'	(1 2 3)	[2,3,1]
r'*r'	(1 3 2)	[3,1,2]
s'	(1 2)	[2,1,3]
s'*r'	(1 3)	[3,2,1]
s'*r'*r'	(2 3)	[1,3,2]

S3 est généré par deux éléments: $r' = (1\,2\,3)$ et $s' = (1\,2)$

Correspondance exacte des générateurs et des décompositions.

I - Implémentation pour le cas général

Etude d'un cas concret



f: S3 → D3
r' = (1 2 3) → r
s' = (1 2) → s

s'étend en un isomorphisme de S3 dans D3

Pour calculer l'image de $(1\ 3\ 2)$:

$$f((1\ 3\ 2)) = f(r' \circ r') = f(r') \circ f(r') = r \circ r = r_{240^\circ}$$

I - Implémentation pour le cas général

L'approche de Tarjan

SONTISOMORPHES

Entrée : Deux groupes G et G' de même ordre, $F \subset G$ une famille de générateurs.

Sortie : (VRAI, ϕ) si G et G' sont isomorphes, FAUX sinon.

Pour chaque application injective $f : F \to G'$:
  Étendre f en une application $\phi : G \to G'$
  Si ESTISOMORPHISME(ϕ, G, G') :
    Renvoyer (VRAI, ϕ)
Renvoyer FAUX

I - Implémentation pour le cas général

L'approche de Tarjan

SONTISOMORPHES

Entrée : Deux groupes G et G' de même ordre, $F \subset G$ une famille de générateurs.

Sortie : (VRAI, ϕ) si G et G' sont isomorphes, FAUX sinon.

Pour chaque application injective $f : F \to G'$:
  Étendre f en une application $\phi : G \to G'$
  Si ESTISOMORPHISME(ϕ, G, G') :
    Renvoyer (VRAI, ϕ)
Renvoyer FAUX

Avec: $m = |F|,\ n = |G| = |G'|$

Nombre d'applications injectives de F dans G : $\dfrac{n!}{(n-m)!}$

Étendre f en $\phi : G \to G' : O(nm)$
Vérifier l'isomorphisme : $O(n^2)$
$\Big\}\ O(n^2)$

I - Implémentation pour le cas général

L'approche de Tarjan

SONTISOMORPHES

Entrée : Deux groupes G et G' de même ordre, $F \subset G$ une famille de générateurs.

Sortie : (VRAI, ϕ) si G et G' sont isomorphes, FAUX sinon.

Pour chaque application injective $f : F \to G'$:
  Etendre f en une application $\phi : G \to G'$
  Si ESTISOMORPHISME(ϕ, G, G') :
    Renvoyer (VRAI, ϕ)
Renvoyer FAUX

Coûte
$O(nm) = O(n^2)$

Gain d'un facteur
$(n-m)!$

Avec: $m = |F|,\ n = |G| = |G'|$

Nombre d'applications injectives de F dans G : $\dfrac{n!}{(n-m)!}$

$$\left.\begin{array}{l} \text{Étendre } f \text{ en } \phi : G \to G' : O(nm) \\ \text{Vérifier l'isomorphisme} : O(n^2) \end{array}\right\} O(n^2)$$

But: trouver une famille de générateurs de petite taille.

I - Implémentation pour le cas général

i) Trouver une famille de générateurs

GÉNÉRATEURS

Entrée : Un groupe fini G d'ordre n.

Sortie : Une famille F de générateurs avec $|F| = O(\log n)$ et R un tableau de listes chainées des décompositions.

$F \leftarrow \emptyset$
$H \leftarrow \emptyset$
Tant que $H \neq G$:
  $x \leftarrow$ un élément de $H \setminus G$
  $F \leftarrow F \cup \{x\}$
  Pour $y \in H$:
    $H \leftarrow H \cup \{x \cdot y\}$
    R[xy] $\leftarrow x \cdot R[y]$
Renvoyer F, R

H double à chaque étape $\Rightarrow |F| = O(log\, n)$

Complexité: $O(nlog(n)))$

I - Implémentation pour le cas général

ii) Complexité finale

SONTISOMORPHES

Entrée : Deux groupes G et G' de même ordre, $F \subset G$ une famille de générateurs.

Sortie : (VRAI, ϕ) si G et G' sont isomorphes, FAUX sinon.

Pour chaque application injective $f : F \to G'$:
  Étendre f en une application $\phi : G \to G'$
  Si ESTISOMORPHISME(ϕ, G, G') :
    Renvoyer (VRAI, ϕ)
Renvoyer FAUX

Complexité pour étendre f en ϕ : $O(n \times |F|) = O(nlog(n))$

Complexité de EstIsomorphisme : $O(n^2)$

Complexité totale d'un passage de boucle: $O(n^2)$

Nombre d'applications $f : G \to G'$ injectives : $O\left(\frac{n!}{(n - \lfloor log(n) \rfloor)!}\right)$

I - Implémentation pour le cas général

iii) Complexité finale

Après calculs, on trouve, avec la formule de Stirling: $n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$

$$O\left(\frac{n!}{(n-\lfloor log(n) \rfloor)!}\right) = O(n^{log(n)+2.45})$$

Complexité finale: $n^{log(n)+O(1)}$

I - Implémentation pour le cas général

Implémentation: structure de groupe

```
13  typedef struct group {
14      char *name; // Nom
15      int n; // Cardinal du groupe
16      int **table; /* Table de Cayley:
17                      le groupe est assimilé
18                      à [0, 1, ..., n-1]
19                  */
20  } group_t;
```

Espace mémoire: $O(n^2)$

Calcul: $O(1)$

I - Implémentation pour le cas général

Implémentation: structures pour les calculs

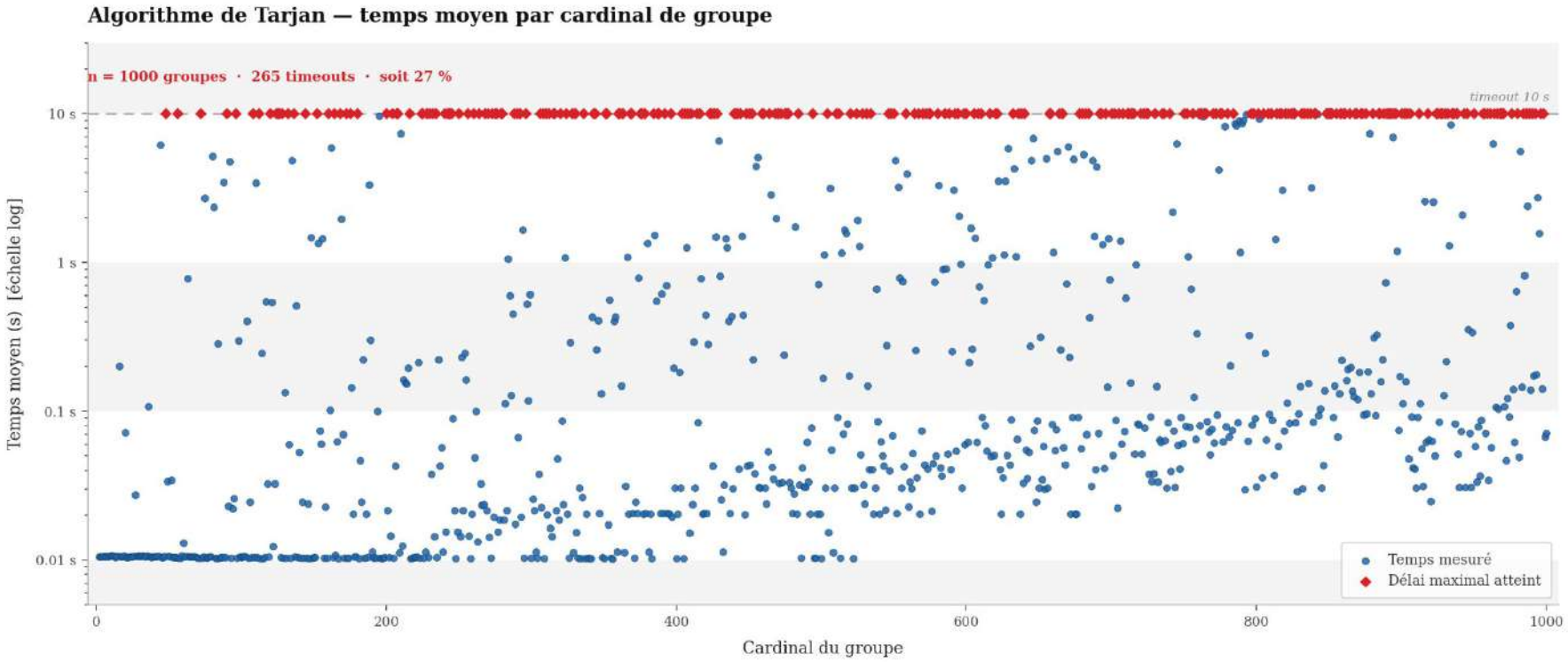
```c
typedef struct node {
    int val;
    struct node* next;
} node_t;

typedef struct list {
    node_t* head;
    node_t* tail;
    int length;
} list_t;
```

```c
typedef struct gentabl {
  int size;
  int num_elts;
  list_t** decomps; /* Table de décomposition des
                       éléments du groupe
                       selon les générateurs
                    */
} gentabl_t;
```

```c
typedef struct group_data {
  gentabl_t* rel; // Table des décompositions
  list_t* gen; // Liste des générateurs
} gdata_t;
```

I - Implémentation pour le cas général

Résultats expérimentaux

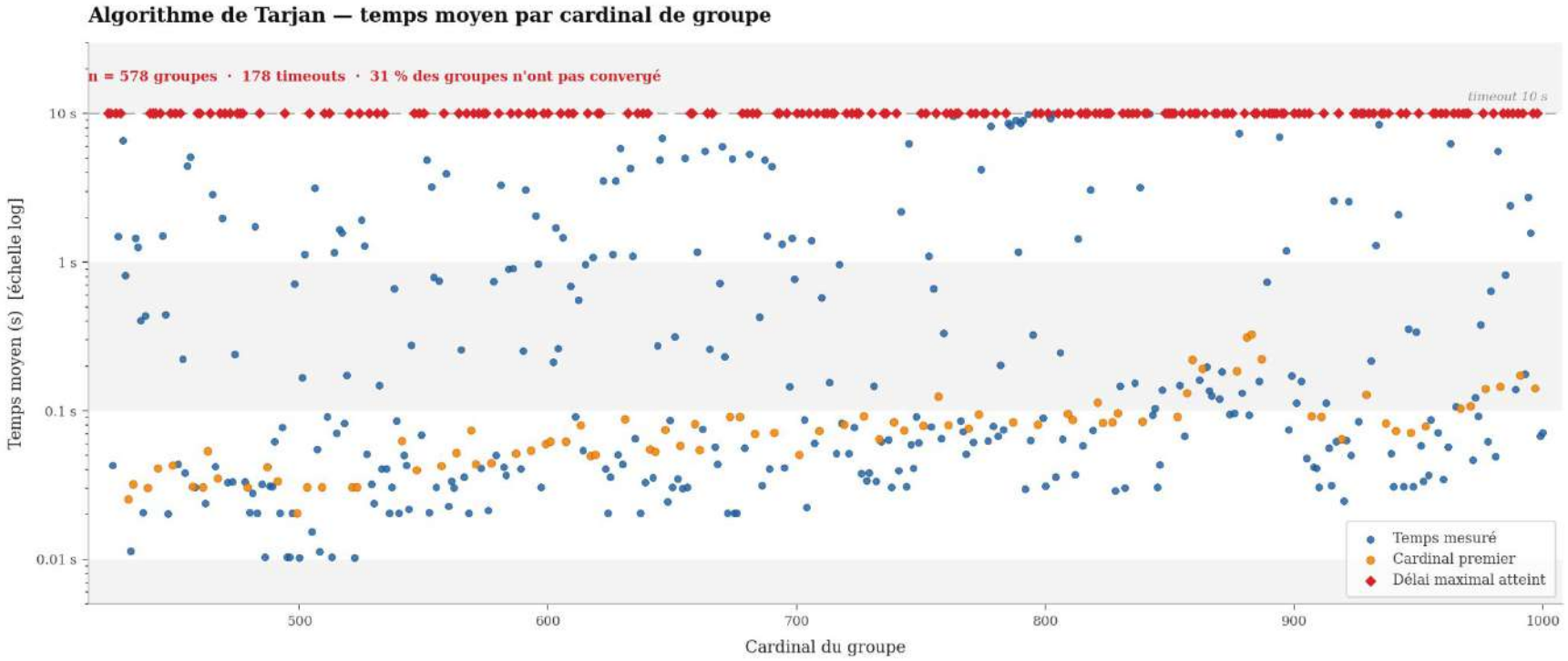


Bleu: temps moyen mesuré (sur dix itérations)

Rouge: délai maximal de 10 secondes atteint - 265 dépassements sur 1000 groupes soit 27%

I - Implémentation pour le cas général

Résultats expérimentaux



Bleu: temps moyen mesuré (sur dix itérations)

Rouge: délai maximal de 10 secondes atteint - 265 dépassements sur 1000 groupes soit 27%

Orange: groupe de cardinal premier - pas très efficace même quand le groupe est cyclique

Transition

II - Le cas particulier des groupes abéliens: l'approche de Nader H. Bshouty

II - Le cas particulier des groupes abéliens

Résultat théorique important.

Théorème de structures des groupes abéliens finis:

Soit G un groupe abélien fini. Alors G est isomorphe à un unique produit de groupes cycliques:

$$G \cong \mathbb{Z}/d_1\mathbb{Z} \times \ldots \times \mathbb{Z}/d_r\mathbb{Z}$$

où les d_i vérifient: $d_1 \mid \ldots \mid d_r$ et les $d_i > 1$

Objectif: trouver les (d_i)

II - Le cas particulier des groupes abéliens

i) Trouver des générateurs et les décompositions

CalculGénérateurs

Entrée : Un groupe fini G avec $n = |G|$.

Sorties : Les générateurs $\{a_1, \ldots, a_t\}$ avec $t \leq log_2(n)$ et une table de listes chainées $(\lambda(x))_{x \in G}$ des décompositions selons les générateurs

---

$A_0 \leftarrow \emptyset$
$G_0 \leftarrow \{e\}$
$i \leftarrow 0$
$\forall a \in G, \lambda(a) \leftarrow$ NULL
Tant que $G \neq G_i$:
  $i \leftarrow i + 1$
  $G_i \leftarrow G_{i-1}$
  Choisir $a_i \in G \setminus G_{i-1}$
  $A_i \leftarrow A_{i-1} \cup \{a_i\}$
  Trouver le plus petit k_i tel que $a_i^{k_i} \in G_{i-1}$
  Pour $(x, j) \in G_{i-1} \times \{1, \ldots k_i - 1\}$:
    $\lambda(a_i^j x) \leftarrow$ CopierListe$(\lambda(x))$
    Ajouter en tête de $\lambda(a_i^j x)$ le nœud (a_i, j, NULL)
    $G_i \leftarrow G_i \cup \{a_i^j x\}$
Renvoyer $t \leftarrow i$, $A \leftarrow A_i$, $(k_1, k_2, \ldots, k_t)$ et la table de listes chaînées λ

Complexité: $O(nlog(n))$

II - Le cas particulier des groupes abéliens

i) Trouver des générateurs et les décompositions

```c
typedef struct group {
  char* name;
  int n;
  int** table;
} group_t;
typedef group_t* group;
```

```c
typedef struct node {
  int val;
  struct node* next;
} node_t;

typedef struct list {
  node_t* head;
  node_t* tail;
  int length;
} list_t;
typedef list_t* list;
```

```c
typedef struct data_s {
  int g;
  int p;
} data_t;
typedef data_t* data;

typedef struct node_data_s {
  int g;
  int p;
  struct node_data_s* next;
} node_data_t;

typedef node_data_t* node_data;

typedef struct data_list_s {
  node_data head;
  node_data tail;
  int length;
} data_list_t;
typedef data_list_t* data_list;
```

```c
typedef struct group_data_s {
  int n;
  bool initialised;

  bool* contains;
  list grp_elts;

  int* order;
  // (at,kt)->...->(a1,k1)
  data_list generators;

  data_list* decomps;
} group_data_t;
typedef group_data_t* group_data;
```

II - Le cas particulier des groupes abéliens

i) Trouver des générateurs et les décompositions

```
Groupe Z/4Z x Z/2Z d'ordre 8
0 1 2 3 4 5 6 7
1 0 3 2 5 4 7 6
2 3 4 5 6 7 0 1
3 2 5 4 7 6 1 0
4 5 6 7 0 1 2 3
5 4 7 6 1 0 3 2
6 7 0 1 2 3 4 5
7 6 1 0 3 2 5 4
Elément neutre: 0

Décompositions:
1: (1,1) -> END
Générateur(s): (1,2) -> END
Nombre d'éléménts: 2
1 -> 0 -> NULL
```

```
Décompositions:
1: (1,1) -> END
2: (3,1) -> (1,1) -> END
3: (3,1) -> END
4: (3,2) -> END
5: (3,2) -> (1,1) -> END
6: (3,3) -> (1,1) -> END
7: (3,3) -> END
Générateur(s): (3,4) -> (1,2) -> END
Nombre d'éléménts: 8
3 -> 2 -> 4 -> 5 -> 7 -> 6 -> 1 -> 0 -> NULL
```

II - Le cas particulier des groupes abéliens

ii) Calculs des relations triangulaires

En notant $H = \{a_1, \ldots, a_t\}$ et en assimilant
la liste chainée $\lambda(x)$ à un tableau $(\lambda_1(x), \ldots, \lambda_t(x))$, on a:

pour i allant de 2 à t: $a_i^{k_i} = a_{i-1}^{l_{i,i-1}} \ldots a_1^{l_{i,1}},\ a_1^{k_1} = e$

$$\forall\, x \in G : x = a_1^{\lambda_1(x)} \ldots a_t^{\lambda_t(x)}$$

On dipose d'un isomorphsme naturel entre G et $\Gamma = \left(\bigoplus_{i=1}^{t} \mathbb{Z}x_i \middle\backslash R \right)$

avec $\Phi : a_1^{\lambda_1} \ldots a_t^{\lambda_t} \to \lambda_1 x_1 + \ldots + \lambda_t x_t$

Où: $R = (k_i x_i = k_{i-1} x_{i-1} + \ldots + k_1 x_1$ pour i allant de 2 à t, $k_1 x_1 = 0)$

II - Le cas particulier des groupes abéliens

iii) Réductions des relations

$$R = \begin{bmatrix} -k_t & \ell_{t,t-1} & \ell_{t,t-2} & \cdots & \ell_{t,2} & \ell_{t,1} \\ 0 & -k_{t-1} & \ell_{t-1,t-2} & \cdots & \ell_{t-1,2} & \ell_{t-1,1} \\ 0 & 0 & -k_{t-2} & \cdots & \ell_{t-2,2} & \ell_{t-2,1} \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 0 & -k_1 \end{bmatrix}$$

Forme normale de Smith: $URV = diag(d_1, \ldots, d_r, 0, \ldots, 0) = D$

En notant h $= \begin{pmatrix} x_t \\ \vdots \\ x_t \end{pmatrix}$ on a:

$$Rh = 0 \iff U^{-1}DV^{-1}h = 0 \iff D(V^{-1}h) = 0$$

II - Le cas particulier des groupes abéliens

ii) Réductions des relations

En notant $y = \begin{pmatrix} y_1 \\ \vdots \\ y_t \end{pmatrix}$ on a:

$$Dy = 0 \iff d_i y_i = 0 \text{ pour i allant de 1 à r}$$

Les (y_i) forment une base de Γ et on en déduit une base de G.

$$G \cong \mathbb{Z}/d_1\mathbb{Z} \times \ldots \times \mathbb{Z}/d_r\mathbb{Z}$$

II - Le cas particulier des groupes abéliens

Complexité totale

Calculs de la FNS - Kanna-Achim Bachem (1979):

pour $A \in M_n(\mathbb{Z})$, et $\sup_{i,j} |a_{i,j}| \leq K$

$$O(n^4 log(K))$$

Inversion d'une matrice de taille $n \times n$ à coefficients entiers: $O(n^3)$

II - Le cas particulier des groupes abéliens

Complexité totale

Calculs de la FNS - Kanna-Achim Bachem (1979):

pour $A \in M_n(\mathbb{Z})$, et $\sup_{i,j} |a_{i,j}| \leq K$

$$O(n^4 log(K))$$

Inversion d'une matrice de taille $n \times n$ à coefficients entiers: $O(n^3)$

Or ici, la matrice est de taille en m x m avec m = O(log n): tous les calculs sont donc en poly(log n).

II - Le cas particulier des groupes abéliens

Complexité totale

Pour décider si deux groupes sont isomorphes:

1 - Calculs des générateurs/relations

$$O(nlog(n))$$

$+$

2 - Réductions des relations: SNF*

$$O(log^6(n))$$

$+$

3 - Comparer les matrices diagonales

$$O(log(n))$$

$$= \quad O(nlog(n))$$

Pour calculer explicitement un isomorphisme:

$+$

4 - Inverser la matrice V

$$O(log^3(n))$$

$+$

5 - Calcul de la base

$$O(log^2(n))$$

$+$

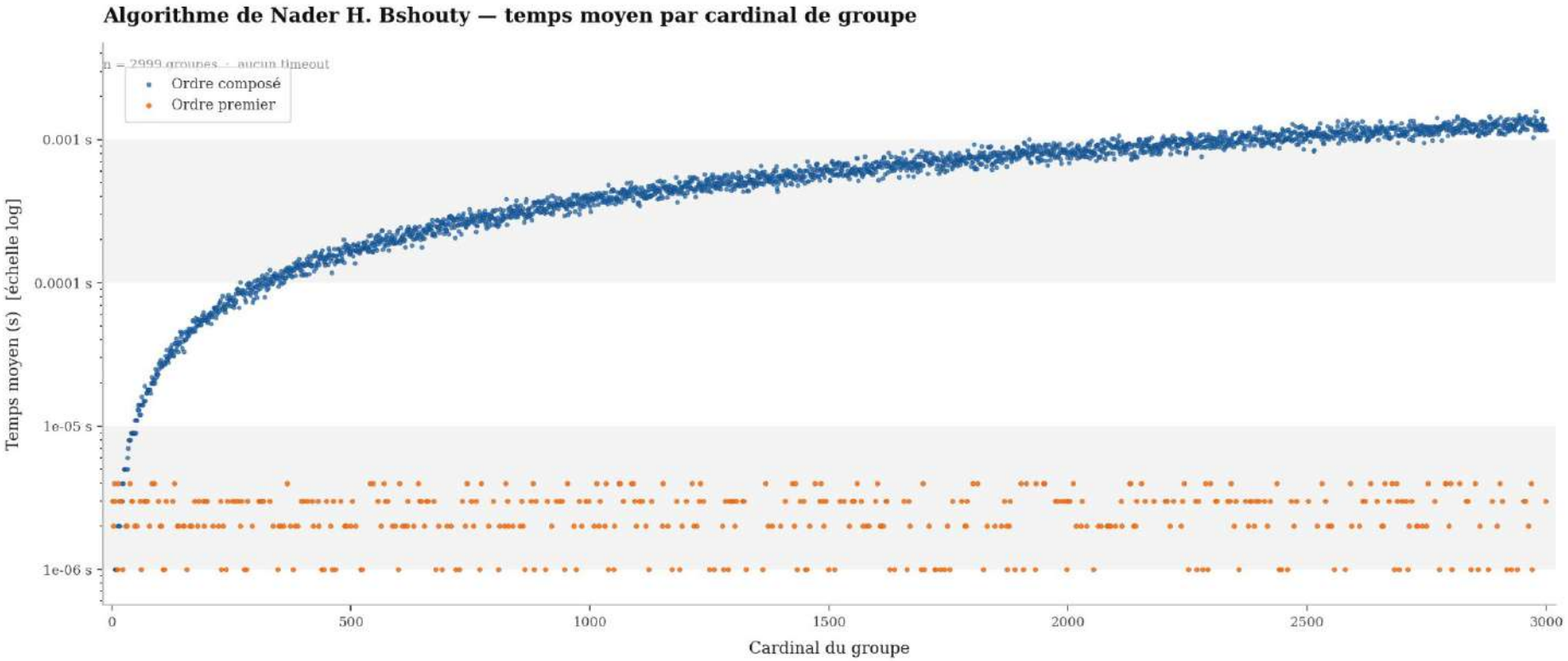
6 - Déterminer l'isomorphisme

$$O(nlog^2(n))$$

$$= O(nlog^2(n))$$

II - Le cas particulier des groupes abéliens

Résultats expérimentaux



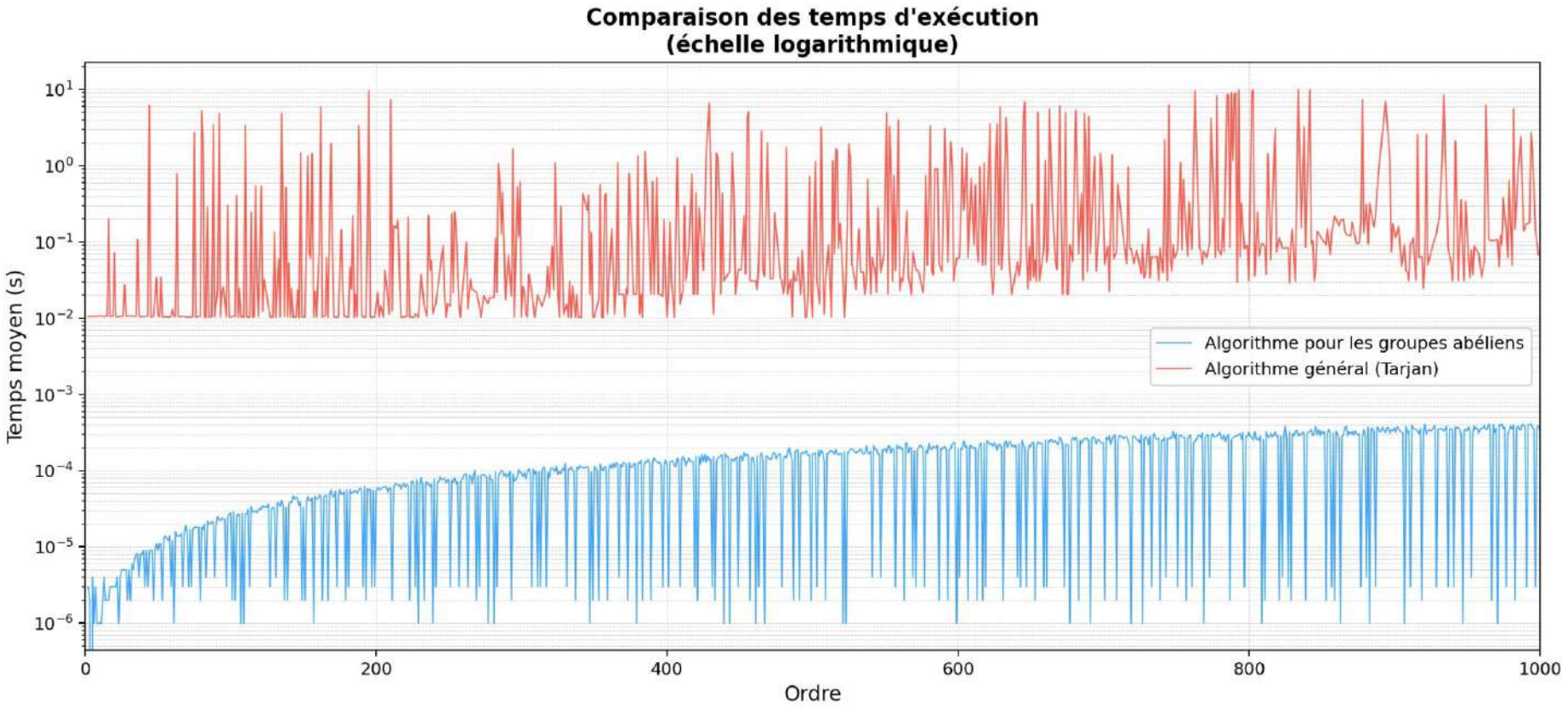
Orange: groupe de cardinal premier, quasi instantané

Transition

III - Conclusion et analyse des résultats

III - Analyse des résultats

Temps d'exécution



Algorithme général: beaucoup d'oscillations

Algorithme pour les groupes abéliens: beaucoup moins d'oscillations

Transition

Annexe: Amélioration de l'algorithme de Tarjan

Annexe: amélioration de l'algorithme de Tarjan

Complexité totale

Pour décider si deux groupes sont isomorphes:

1 - Calculs des générateurs/relations

$O(nlog(n))$

$+$

2 - Réductions des relations: SNF*

$O(log^6(n))$

$+$

3 - Comparer les matrices diagonales

$O(log(n))$

$$= \quad O(nlog(n))$$

Annexe: amélioration de l'algorithme de Tarjan

Complexité totale

Pour décider si deux groupes sont isomorphes:

1 - Calculs des générateurs/relations

$O(nlog(n))$

+

2 - Réductions des relations: SNF*

$O(log^6(n))$

+

3 - Comparer les matrices diagonales

$O(log(n))$

$= \quad O(nlog(n))$

Choisir $a_i \in G \setminus G_{i-1}$
$A_i \leftarrow A_{i-1} \cup \{a_i\}$
Trouver le plus petit k_i tel que $a_i^{k_i} \in G_{i-1}$
Pour $(x, j) \in G_{i-1} \times \{1, \dots k_i - 1\}$:
$\lambda(a_i^j x) \leftarrow$ CopierListe$(\lambda(x))$
Ajouter en tête de $\lambda(a_i^j x)$ le nœud (a_i, j, NULL)

Coût: O(log(n))

Annexe: amélioration de l'algorithme de Tarjan

Solution

CalculGénérateurs

Entrée : Un groupe fini G avec $n = |G|$.

Sorties : Les générateurs $\{a_1, \ldots, a_t\}$ avec $t \leq log_2(n)$ et une table de listes chainées $(\lambda(x))_{x \in G}$ des décompositions selons les générateurs

---

$A_0 \leftarrow \emptyset$
$G_0 \leftarrow \{e\}$
$i \leftarrow 0$
$\forall a \in G, \lambda(a) \leftarrow$ NULL
Tant que $G \neq G_i$:
  $i \leftarrow i + 1$
  $G_i \leftarrow G_{i-1}$
  Choisir $a_i \in G \setminus G_{i-1}$
  $A_i \leftarrow A_{i-1} \cup \{a_i\}$
  Trouver le plus petit k_i tel que $a_i^{k_i} \in G_{i-1}$
  Pour $(x, j) \in G_{i-1} \times \{1, \ldots k_i - 1\}$:
    ~~$\lambda(a_i^j x) \leftarrow$ CopierListe($\lambda(x)$)~~
    $\lambda(a_i^j x) \leftarrow$ nouveau nœud(a_i, j, $next \rightarrow \lambda(x)$)
    $G_i \leftarrow G_i \cup \{a_i^j x\}$
Renvoyer $t \leftarrow i$, $A \leftarrow A_i$, $(k_1, k_2, \ldots, k_t)$ et la table de listes chaînées λ

Coût:
~~**O(log(n))**~~
O(1)

Compléxité: $O(n)$

Annexe: amélioration de l'algorithme de Tarjan

Complexité totale

Pour décider si deux groupes sont isomorphes:

1 - Calculs des générateurs/relations

~~$O(nlog(n))$~~

$O(n)$

$+$

2 - Réductions des relations: SNF*

$O(log^6(n))$

$+$

3 - Comparer les matrices diagonales

$O(log(n))$

$= \quad$ ~~$O(nlog(n))$~~

$O(n)$

On peut décider si deux groupes abéliens, finis, sont isomorphes en temps linéaire.

Annexe:
Code source

```c
#ifndef GEN_TBL
#define GEN_TBL

#include "linked_list.h"

typedef struct gentabl {
 int size;
 int num_elts;
 list_t** decomps; /* Table de décomposition des
            éléments du groupe
            selon les générateurs
          */
} gentabl_t;

gentabl_t* create_empty_gentabl(int n);

bool is_empty_gentabl(gentabl_t* gt);

void clear_gentabl(gentabl_t* gt);

bool is_empty_index(gentabl_t* gt, int index);

list_t* get_copy_index_val(gentabl_t* gt, int i);

void copy_at_index(gentabl_t* gt, int index, list_t* val);

void del_index(gentabl_t* gt, int index);

void show_all_gentabl(gentabl_t* gt);

void free_gentabl(gentabl_t* gt);

#endif
```

```c
#include "gen_table.h"
#include "linked_list.h"

gentabl_t* create_empty_gentabl(int n) {
 gentabl_t* gt = malloc(sizeof(gentabl_t));
 gt->num_elts = 0;
 gt->size = n;

 gt->decomps = malloc(gt->size*sizeof(list_t*));
 for (int i = 0; i < gt->size; i++) {
  gt->decomps[i] = create_empty_list();
 }
 return gt;
}

bool is_empty_gentabl(gentabl_t* gt) {
 assert(gt != NULL);
 return (gt->size == 0);
}

void clear_gentabl(gentabl_t* gt) {
 for (int i = 0; i < gt->size; i++) {
  free_list(gt->decomps[i]);
 }
 free(gt->decomps);
 free(gt);
}

bool is_empty_index(gentabl_t* gt, int index) {
 assert(index < gt->size);
 list_t* to_check = gt->decomps[index];
 assert(to_check != NULL);
 return (is_empty(to_check));
}

list_t* get_copy_index_val(gentabl_t* gt, int i) {
 assert(!is_empty_index(gt, i));
 list_t* copy = deep_copy(gt->decomps[i]);
 return copy;
}

void copy_at_index(gentabl_t* gt, int index, list_t* val) {
 assert(gt != NULL);
 assert(index < gt->size);
 if (gt->decomps[index] != NULL) {
  free_list(gt->decomps[index]);
 }

 gt->decomps[index] = deep_copy(val);
}

void del_index(gentabl_t* gt, int index) {
 assert(gt != NULL);
 assert(index < gt->size);
 list_t* to_free = gt->decomps[index];
 free_list(to_free);
 gt->decomps[index] = create_empty_list();
}

void show_all_gentabl(gentabl_t* gt) {
 for (int i = 0; i < gt->size; i++) {
  printf("[%d]: ", i);
  show_values(gt->decomps[i]);
  printf("\n");
 }
}
```

```
void free_gentabl(gentabl_t* gt) {
 assert(gt != NULL);
 for (int i = 0; i < gt->size; i++) {
  free_list(gt->decomps[i]);
 }
 free(gt->decomps);
 free(gt);
}
```

```c
#ifndef GROUP_H
#define GROUP_H

#include "linked_list.h"
#include "gen_table.h"
#include <assert.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

typedef struct group {
  char *name; // Nom
  int n; // Cardinal du groupe
  int **table; /* Table de Cayley:
                  le groupe est assimilé
                  à [0, 1, ..., n-1]
               */
} group_t;

typedef struct group_data {
  gentabl_t* rel; // Table des décompositions
  list_t* gen; // Liste des générateurs
} gdata_t;

typedef struct decomps_abelian_s {
  int n;
  int num_basis;
  int* basis;
  int** basis_decomps;
} decomps_t;

void add_sum_to_decomp(int x, int y, decomps_t* d);

group_t *group_empty(int n);
void group_name(group_t *g, char *s);
void group_free(group_t *g);
void group_print(group_t *g);

gdata_t* create_empty_gdata();
gdata_t* parse_entries(list_t* gen, gentabl_t* rel);
void free_gdata(gdata_t* gdata);

list_t* get_generator(gdata_t* gdata);
gentabl_t* get_rel(gdata_t* gdata);

group_t *from_matrix(int **matrix);
group_t *copy_group(group_t *g);

int calculate(group_t* g, int x, int y);
bool is_neutral(group_t* g, int x);
int get_neutral(group_t* g);
int calculate_order(group_t* g, int x);
int* get_orders_list(group_t* g);

int calc_generated_subgroup(group_t* g, int gen, list_t* generators, gentabl_t* decomps);
gdata_t* calc_gen_and_rel(group_t* g);
list_t* get_cyclic_subgroup_element(group_t* g, int x, int e);
bool are_isomorphic(group_t* g1, group_t* g2);
bool are_isomorphic_with_order(group_t* g1, group_t* g2);

void print_array(int* arr, int n);

bool are_isomorphic_with_order_custom_gen(group_t* g1, group_t* g2, int* generators, int
num_gen);
```

```
bool are_isomorphic_probabilistic(group_t* g1, group_t* g2, int max_attempts);
bool are_isomorphic_hybrid(group_t* g1, group_t* g2, long long deterministic_threshold, int
probabilistic_attempts);
bool are_isomorphic_adaptive(group_t* g1, group_t* g2);
#endif
```

```c
#include <assert.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "group.h"
#include "maps.h"
#include "linked_list.h"

int *zero_array(int n) {
 int *res = malloc(n * sizeof(int));
 for (int i = 0; i < n; i++) {
  res[i] = 0;
 }
 return res;
}

void group_name(group_t *g, char *s) {
 g->name = s;
}

void add_sum_to_decomp(int x, int y, decomps_t* d) {
 int* x_decomp = d->basis_decomps[x];
 int* y_decomp = d->basis_decomps[y];
 int z = (x+y) % d->n;
 int* z_decomp = d->basis_decomps[z];
 for (int i = 0; i < d->n; i++) {
  if (x_decomp[i] == -1 || y_decomp[i] == -1) {
   z_decomp[i] = -1;
  } else {
   z_decomp[i] = (x_decomp[i] + y_decomp[i]) % d->n;
  }
 }
}

group_t *group_empty(int n) {
 group_t *new_empty_group = malloc(sizeof(group_t));
 new_empty_group->n = n;
 new_empty_group->table = malloc(n * sizeof(int *));
 for (int i = 0; i < n; i++) {
  new_empty_group->table[i] = zero_array(n);
 }
  return new_empty_group;
}

void group_free(group_t *g) {
 assert(g != NULL && "[-] Pointeur nul.\n");
  int n = g->n;
 for (int i = 0; i < n; i++) {
  free(g->table[i]);
 }
 free(g->table);
 free(g);
}


gdata_t* create_empty_gdata() {
 gdata_t* gdata = malloc(sizeof(gdata_t));
 return gdata;
}

// WE GIVE THE POINTERS WE DON'T COPY.
gdata_t* parse_entries(list_t* gen, gentabl_t* rel) {
 gdata_t* gdata = create_empty_gdata();
 gdata->gen = gen;
 gdata->rel = rel;
 return gdata;
```

```
}

void free_gdata(gdata_t* gdata) {
 assert(gdata != NULL);
 free_list(gdata->gen);
 free_gentabl(gdata->rel);
 free(gdata);
}

list_t* get_generator(gdata_t* gdata) {
 assert(gdata != NULL);
 return gdata->gen;
}

gentabl_t* get_rel(gdata_t* gdata) {
 assert(gdata != NULL);
 return gdata->rel;
}

void group_print(group_t *g) {
 assert(g != NULL && "[-] Pointeur nul.\n");
  int n = g->n;
 if (g->name == NULL || g->name[0] == '\0') {
  printf("Groupe d'ordre: %d\n", n);
 } else {
  printf("Groupe %s d'ordre %d\n", g->name, n);
 }
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   printf("%d ", g->table[i][j]);
  }
  printf("\n");
 }
}

group_t *from_matrix(int **m) {
 assert(m != NULL && "[-] Pointeur nul.\n");
 int n = sizeof(*m[0]) / sizeof(int);
 group_t *res = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   res->table[i][j] = m[i][j];
  }
 }
 return res;
}

group_t *copy_group(group_t *g) {
 assert(g != NULL && "[-] Pointeur nul.\n");
 int n = g->n;
 group_t *copy = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   copy->table[i][j] = g->table[i][j];
  }
 }
 return copy;
}

int calculate(group_t* g, int x, int y) {
 if (x == -1) {
  return y;
 }
 return g->table[x][y];
}

bool is_neutral(group_t* g, int x) {
```

```
 assert(g != NULL);
 assert(x >= 0 && x < g->n);
 int res;
 if (x < g->n - 1) {
  res = calculate(g, x, x+1);
  return (res == x+1);
 } else {
  res = calculate(g, x, 0);
  return (res == 0);
 }
}

int get_neutral(group_t* g) {
 assert(g != NULL);
 for (int i = 0; i < g->n; i++) {
  if (is_neutral(g, i)) {
   return i;
  }
 }
 printf("Error\n.");
 return -1;
}

int calculate_order(group_t* g, int x) {
 assert(g != NULL);
 assert(x >= 0 && x < g->n);
 if (is_neutral(g, x)) {
  return 0;
 }
 int curr = x;
 int order = 1;
 while (!is_neutral(g, curr)) {
  curr = calculate(g, curr, x);
  order++;
 }

 return order;
}

int* get_orders_list(group_t* g) {
 assert(g != NULL);
 int* orders = malloc(g->n*sizeof(int));
 for (int x = 0; x < g->n; x++) {
  orders[x] = calculate_order(g, x);
 }
 return orders;
}

list_t* get_cyclic_subgroup_element(group_t* g, int x, int e) {
 assert(g != NULL);
 assert(0 <= x < g->n);
 assert(is_neutral(g, e));
 int curr = x;
 list_t* cyclic = create_empty_list();
 push(cyclic, e);
 while (!is_neutral(g, curr)) {
  push(cyclic, curr);
  curr = calculate(g, curr, x);
 }

 return cyclic;
}

int calc_generated_subgroup(group_t* g, int new_gen, list_t* generators, gentabl_t*
decomp_tbl) {
 assert(decomp_tbl->size = g->n);
 assert(generators != NULL && g != NULL);
```

```
 if (new_gen >= g->n) {
  return -1;
 }

 push(generators, new_gen);

 list_t* single = create_single(new_gen);
 copy_at_index(decomp_tbl, new_gen, single);
 free_list(single);

 // Variable pour le candidat d'un autre générateur
 int next_gen = new_gen+1;

 int to_add;
 list_t* to_add_decomp;
 for (int elt = 0; elt < g->n; elt++) {
  if (!is_empty_index(decomp_tbl, elt)) {
   to_add = calculate(g, elt, new_gen);
   if (is_empty_index(decomp_tbl, to_add)) {
    // Calculer la décomposition de to_add.
    to_add_decomp = get_copy_index_val(decomp_tbl, elt);
    append(to_add_decomp, new_gen);
    copy_at_index(decomp_tbl, to_add, to_add_decomp);

    free_list(to_add_decomp);
   }
   if (next_gen < g->n && !is_empty_index(decomp_tbl, next_gen)) {
    next_gen++;
   }
  }
 }

 return next_gen;
}

gdata_t* calc_gen_and_rel(group_t* g) {
 list_t* generators = create_empty_list();
 gentabl_t* relations = create_empty_gentabl(g->n);

 int curr_gen = 0;
 while (curr_gen < g->n) {
  curr_gen = calc_generated_subgroup(g, curr_gen, generators, relations);
 }

 gdata_t* result = parse_entries(generators, relations);
 return result;
}

void print_array(int* arr, int n) {
 for (int i = 0; i < n; i++) {
  printf("%d ", arr[i]);
 }
 printf("\n");
}

bool are_isomorphic(group_t* g1, group_t* g2) {
 assert(g1 != NULL && g2 != NULL);
 assert(g1->n == g2->n);

 // Précalculs sur les données de g1.
 gdata_t* g1_gdata  = calc_gen_and_rel(g1);
 gentabl_t* rel = g1_gdata->rel;
 list_t* gen = g1_gdata->gen;

 int n = g1->n;
 int m = gen->length;
```

```
 int count_perm = 0;
 int* perm;
 // Nombres de permutations.
 llong num_perm = factorial(m);

 // Table des coefficients binomiaux: binom[k][p] = k parmi p
 int** binom = get_binomial(n);

 int count = 0;
 int curr_subset_num = 0;
 int num_subsets = binom[m][n];
 int* induced_map;
 bool found = false;

 while (curr_subset_num < num_subsets && !found) {
  int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
  print_array(chosen_images, m);
  int* curr_permutated_images;
  int curr_perm_num = 0;
  while (curr_perm_num < num_perm && !found) {
   perm = get_kth_permutation_rec(m, curr_perm_num);
   curr_permutated_images = get_from_permutation(perm, chosen_images, m);
   induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
   if (is_isomorphism(g1, g1_gdata, g2, induced_map, n)) {
    found = true;
    printf("[+] Isomorphisme trouvé.\n");
    print_map(induced_map, n);

    free(induced_map);
    free(perm);
    free(curr_permutated_images);
    break;
   }

   free(induced_map);
   free(perm);
   free(curr_permutated_images);
   curr_perm_num++;
   count++;
  }

  free(chosen_images);
  curr_subset_num++;
 }

 free_gdata(g1_gdata);
 free_matrix(binom, n);

 return found;

}

bool are_isomorphic_with_order(group_t* g1, group_t* g2) {
 assert(g1 != NULL && g2 != NULL);
 assert(g1->n == g2->n);

 gdata_t* g1_gdata  = calc_gen_and_rel(g1);
 gentabl_t* rel = g1_gdata->rel;
 int* orders_g1 = get_orders_list(g1);
 int* orders_g2 = get_orders_list(g2);

 list_t* gen = g1_gdata->gen;

 int n = g1->n;
 int m = gen->length;
```

```
 int count_perm = 0;
 int* perm;
 llong num_perm = factorial(m);

 int** binom = get_binomial(n);

 int count = 0;

 int curr_subset_num = 0;
 int num_subsets = binom[m][n];
 int* induced_map;
 bool found = false;

 while (curr_subset_num < num_subsets && !found) {
  int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
  int* curr_permutated_images;
   int curr_perm_num = 0;
   while (curr_perm_num < num_perm && !found) {
    perm = get_kth_permutation_rec(m, curr_perm_num);
    curr_permutated_images = get_from_permutation(perm, chosen_images, m);
    induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
    if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2, induced_map, n)) {
     found = true;

     free(induced_map);
     free(perm);
     free(curr_permutated_images);
     break;
    }

    free(induced_map);
    free(perm);
    free(curr_permutated_images);
    curr_perm_num++;
    count++;
   }

  free(chosen_images);
  curr_subset_num++;
 }

 free(orders_g1);
 free(orders_g2);
 free_gdata(g1_gdata);
 free_matrix(binom, n);

 return found;
}

bool are_isomorphic_with_order_custom_gen(group_t* g1, group_t* g2, int* generators, int
num_gen) {
 assert(g1 != NULL && g2 != NULL);
 assert(g1->n == g2->n);
 assert(generators != NULL && num_gen > 0);

 list_t* gen = create_empty_list();
 gentabl_t* rel = create_empty_gentabl(g1->n);

 for (int i = 0; i < num_gen; i++) {
  assert(generators[i] >= 0 && generators[i] < g1->n);
  calc_generated_subgroup(g1, generators[i], gen, rel);
 }

 gdata_t* g1_gdata = create_empty_gdata();
 g1_gdata->gen = gen;
 g1_gdata->rel = rel;
```

```
 int* orders_g1 = get_orders_list(g1);
 int* orders_g2 = get_orders_list(g2);

 int n = g1->n;
 int m = gen->length;

 int* perm;
 llong num_perm = factorial(m);

 int** binom = get_binomial(n);

 int count = 0;
 int curr_subset_num = 0;
 int num_subsets = binom[m][n];
 int* induced_map;
 bool found = false;

 while (curr_subset_num < num_subsets && !found) {
  int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
  int* curr_permutated_images;
  int curr_perm_num = 0;

  while (curr_perm_num < num_perm && !found) {
   perm = get_kth_permutation_rec(m, curr_perm_num);
   curr_permutated_images = get_from_permutation(perm, chosen_images, m);
   induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);

   if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2, induced_map, n)) {
    found = true;
    printf("[+] Isomorphisme trouvé.\n");
    print_map(induced_map, n);

    free(induced_map);
    free(perm);
    free(curr_permutated_images);
    break;
   }

   free(induced_map);
   free(perm);
   free(curr_permutated_images);
   curr_perm_num++;
   count++;
  }

  free(chosen_images);
  curr_subset_num++;
 }

 free(orders_g1);
 free(orders_g2);
 free_gdata(g1_gdata);
 free_matrix(binom, n);

 return found;
}

bool are_isomorphic_random(group_t* g1, group_t* g2) {
 assert(g1 != NULL && g2 != NULL);
 assert(g1->n == g2->n);

 gdata_t* g1_gdata  = calc_gen_and_rel(g1);
 gentabl_t* rel = g1_gdata->rel;
 int* orders_g1 = get_orders_list(g1);
 int* orders_g2 = get_orders_list(g2);

 list_t* gen = g1_gdata->gen;
```

```
 int n = g1->n;
 int m = gen->length;

 int count_perm = 0;
 int* perm;
 llong num_perm = factorial(m);

 int** binom = get_binomial(n);

 int count = 0;

 int curr_subset_num = 0;
 int num_subsets = binom[m][n];
 int* induced_map;
 bool found = false;

 while (curr_subset_num < num_subsets && !found) {
  int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
  int* curr_permutated_images;
   int curr_perm_num = 0;
   while (curr_perm_num < num_perm && !found) {
    perm = get_kth_permutation_rec(m, curr_perm_num);
    curr_permutated_images = get_from_permutation(perm, chosen_images, m);
    induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
    if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2, induced_map, n)) {
     found = true;
     printf("[+] Isomorphisme trouvé.\n");
     print_map(induced_map, n);

     free(induced_map);
     free(perm);
     free(curr_permutated_images);
     break;
    }

    free(induced_map);
    free(perm);
    free(curr_permutated_images);
    curr_perm_num++;
    count++;
   }

  free(chosen_images);
  curr_subset_num++;
 }

 free(orders_g1);
 free(orders_g2);
 free_gdata(g1_gdata);
 free_matrix(binom, n);

 return found;
}

bool are_isomorphic_probabilistic(group_t* g1, group_t* g2, int max_attempts) {
 assert(g1 != NULL && g2 != NULL);
 assert(g1->n == g2->n);

 srand(time(NULL));

 gdata_t* g1_gdata = calc_gen_and_rel(g1);
 gentabl_t* rel = g1_gdata->rel;
 list_t* gen = g1_gdata->gen;

 int n = g1->n;
 int m = gen->length;
```

```
 int* orders_g1 = get_orders_list(g1);
 int* orders_g2 = get_orders_list(g2);

 int* gen_orders = malloc(m * sizeof(int));
 int gen_idx = 0;
 for (node_t* c = gen->head; c != NULL; c = c->next) {
  gen_orders[gen_idx++] = orders_g1[c->val];
 }

 bool found = false;
 int* induced_map = NULL;

 for (int attempt = 0; attempt < max_attempts && !found; attempt++) {
  int* chosen_images = get_random_subset(m, n);

  bool order_compatible = true;
  for (int i = 0; i < m; i++) {
   if (gen_orders[i] != orders_g2[chosen_images[i]]) {
    order_compatible = false;
    break;
   }
  }

  if (!order_compatible) {
   free(chosen_images);
   continue;
  }

  int* perm = get_random_permutation(m);
  int* permuted_images = get_from_permutation(perm, chosen_images, m);

  induced_map = get_full_map(g1_gdata, g2, permuted_images, m);

  if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2, induced_map, n)) {
   found = true;
   printf("[+] Isomorphisme trouvé après %d itérations!\n", attempt + 1);
   print_map(induced_map, n);
  }

  free(chosen_images);
  free(perm);
  free(permuted_images);
  if (!found) {
   free(induced_map);
  }

  if ((attempt + 1) % 10000 == 0) {
   printf("Itération %d...\n", attempt + 1);
  }
 }

 free(gen_orders);
 free(orders_g1);
 free(orders_g2);
 free_gdata(g1_gdata);

 if (found) {
  free(induced_map);
 }

 return found;
}

bool are_isomorphic_hybrid(group_t* g1, group_t* g2, long long deterministic_threshold, int
probabilistic_attempts) {
 assert(g1 != NULL && g2 != NULL);
```

```
 assert(g1->n == g2->n);

 gdata_t* g1_gdata = calc_gen_and_rel(g1);
 int n = g1->n;
 int m = g1_gdata->gen->length;
 free_gdata(g1_gdata);

 int** binom = get_binomial(n);
 long long num_subsets = binom[m][n];
 free_matrix(binom, n);

 llong num_perm = factorial(m);
 long long total_candidates = num_subsets * num_perm;

 if (total_candidates <= deterministic_threshold) {

  return are_isomorphic_with_order(g1, g2);
 } else {
  return are_isomorphic_probabilistic(g1, g2, probabilistic_attempts);
 }
}

bool are_isomorphic_adaptive(group_t* g1, group_t* g2) {
 assert(g1 != NULL && g2 != NULL);
 assert(g1->n == g2->n);

 int n = g1->n;
 gdata_t* g1_gdata = calc_gen_and_rel(g1);
 int m = g1_gdata->gen->length;
 free_gdata(g1_gdata);

 int** binom = get_binomial(n);
 long long total_space = binom[m][n] * factorial(m);
 free_matrix(binom, n);

 int attempts = (int)sqrt((double)total_space);
 if (attempts < 1000) attempts = 1000;
 if (attempts > 1000000) attempts = 1000000;

 return are_isomorphic_probabilistic(g1, g2, attempts);
}
```

```c
#ifndef LL
#define LL

#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <stdbool.h>

typedef struct node {
    int val;
    struct node* next;
} node_t;

typedef struct list {
    node_t* head;
    node_t* tail;
    int length;
} list_t;

node_t* new_node(int n);

list_t* create_empty_list();
list_t* create_example(int n);
list_t* create_single(int n);

list_t* deep_copy(list_t* l);

bool is_empty(list_t* linked);

void show_list(list_t* linked);
void show_values(list_t* linked);

void push(list_t* linked, int n);

void append(list_t* linked, int n);

void insert(list_t* linked, int index, int x);

int pop(list_t* linked);

node_t* free_get_next(node_t* node);
void free_list(list_t* linked);

#endif
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <stdbool.h>

#include "linked_list.h"

node_t* new_node(int n) {
    node_t* out = malloc(sizeof(node_t));
    out->next = NULL;
    out->val = n;
    return out;
}

list_t* create_empty_list() {
    list_t* linked = malloc(sizeof(list_t));
    linked->head = NULL;
    linked->tail = NULL;
    linked->length = 0;
    return linked;
}

list_t* create_example(int n) {
    list_t* linked = create_empty_list();
    for (int i = 0; i < n; i++) {
        append(linked, i);
    }
    return linked;
}

list_t* create_single(int n) {
    list_t* linked = create_empty_list();
    push(linked, n);
    return linked;
}

list_t* deep_copy(list_t* l) {
    list_t* copy = create_empty_list();
    node_t* to_append;
    for (node_t* c = l->head; c != NULL; c = c->next) {
        append(copy, c->val);
    }

    return copy;
}

bool is_empty(list_t* linked) {
    return (linked->length == 0);
}

void show_list(list_t* linked) {
    printf("TÄTE: %d\n", linked->head->val);
    printf("QUEUE: %d\n", linked->tail->val);
    printf("LONGUEUR: %d\n", linked->length);

    node_t* curr = linked->head;
    while (curr != NULL) {
        printf("%d -> ", curr->val);
        curr = curr->next;
    }
    printf("FIN\n");
}

void show_values(list_t* linked) {
    for (node_t* c = linked->head; c != NULL; c = c->next) {
        printf("%d -> ", c->val);
    }
```

```
    printf("T\n");
}

void push(list_t* linked, int n) {
    node_t* new_head = malloc(sizeof(node_t));
    new_head->val = n;

    node_t* old_head = linked->head;
    new_head->next = old_head;
    linked->head = new_head;
    linked->length += 1;

    if (linked->length == 1) {
        linked->tail = new_head;
    }
}

void append(list_t* linked, int n) {
    assert(linked != NULL);
    if (linked->length == 0) {
        node_t* new_tail = new_node(n);
        linked->tail = new_tail;
        linked->head = new_tail;
        linked->length++;
    } else {
        node_t* new_tail = malloc(sizeof(node_t));
        new_tail->val = n;
        new_tail->next = NULL;

        node_t* old_tail = linked->tail;
        old_tail->next = new_tail;
        linked->tail = new_tail;
        linked->length += 1;
    }
}

void insert(list_t* linked, int index, int x) {
    assert(index <= linked->length);
    if (index == 0) {
        push(linked, x);
    }
    if (index == linked->length) {
        append(linked, x);
    }
    else {
        node_t* new_node = malloc(sizeof(node_t));
        new_node->val = x;

        node_t* curr = linked->head;
        int i = 0;
        while (i < index-1) {
            curr = curr->next;
            i += 1;
        }
        new_node->next = curr->next;
        curr->next = new_node;

        linked->length += 1;
    }
}

int pop(list_t* linked) {
    assert(is_empty(linked) == false);
    node_t* new_head = linked->head->next;
    node_t* old_head = linked->head;
    int val = old_head->val;
    linked->head = new_head;
```

```
    free(old_head);
    return val;
}

node_t* free_get_next(node_t* node) {
    assert(node != NULL);
    node_t* next = node->next;
    free(node);
    return next;
}

void free_list(list_t* linked) {
    assert(linked != NULL);
    node_t* curr = linked->head;
    while (curr != NULL) {
        curr = free_get_next(curr);
    }
    free(linked);
}
```

```c
#ifndef ALGS_H
#define ALGS_H

#include "../Groups/group.h"
#include "../Gen_Table/gen_table.h"
#include "../Garbage_Collector/garbage_collector.h"
#include "../Linked_List/linked_list.h"
#include "snf.h"
#include "algs.h"
#include <stdbool.h>

int find_neutral(group g);
int fast_exp(group g, int x, int neutral, int n);

int calculate_next_gen_and_rel(group g, group_data gd, int new_gen, int neutral);
int calculate_next_gen_and_rel_fast(group g, group_data gd, gbg_collector coll, int new_gen,
int neutral);

group_data calculate_group_data(group g);
group_data calculate_group_data_fast(group g, gbg_collector trash);

void check_group_data(group g, group_data gd, int neutral);

void print_arr(int* arr, int n);

matrix get_trig_rel_table(group g, int neutral, group_data gd);

typedef struct {
 int n;
 int* order;
 int* elts;
} basis_s;
typedef basis_s* basis;

basis create_empty_basis(int n);
void free_basis(basis b);
void check_basis(group g, basis b, int neutral);
void show_basis(basis b);

basis get_basis_from_data(group g, group_data gd, int neutral, matrix D, matrix V_inv);
basis get_basis_from_rel(group g, group_data gd, int neutral);
basis get_basis(group g);

void iterate_decomps(int* decomps, int dim, int i, int* orders);
bool are_isomorphic(group g1, group g2);
int* calculate_isomorphism(group g1, group g2);

bool check_isomorphism(group g1, group g2, int* iso);
#endif
```

```c
#include "algs.h"

int find_neutral(group g) {
 int next = 0;
 for (int i = 0; i < g->n; i++) {
  next = (i+1) % g->n;
  if (calculate(g, i, next) == next) {
   return i;
  }
 }
 printf("[-] Erreur.\n");
 return -1;
}

int fast_exp(group g, int x, int neutral, int n) {
 assert(g != NULL && x >= 0 && x < g->n);
 assert(is_neutral(g, neutral));
 if (n >= 0) {
  int res = neutral;
  int a = x;
  int p = (n % g->n);
  while (p > 0) {
   if (p % 2 == 1) {
    res = calculate(g, res, a);
   }
   a = calculate(g, a, a);
   p /= 2;
  }

  return res;
 } else {
  int p = (n % g->n) + g->n;
  return fast_exp(g, x, neutral, p);
 }
}

// Calcule les relations étant donné un nouveau générateur.
int calculate_next_gen_and_rel(group g, group_data gd, int new_gen, int neutral) {
 assert(g != NULL && gd != NULL);
 assert(new_gen >= 0);

 if (!gd->initialised) {
  init_empty_group_data(gd, neutral);
 }
 list new_grp_elts = create_empty_list();
 int curr = new_gen;
 int k = 1;
 int to_add, x;

 while (!gd->contains[curr]) {
  for (node_t* node = gd->grp_elts->head; node != NULL; node = node->next) {
   x = node->val;
   to_add = calculate(g, curr, x);

   if (!gd->contains[curr]) {
    push(new_grp_elts, curr);
    push_data_list(gd->decomps[curr], new_gen, k);
    gd->contains[curr] = true;
   }

   if (!gd->contains[to_add]) {
    push(new_grp_elts, to_add);
    del_copy_from(&gd->decomps[to_add], gd->decomps[x]);
    push_data_list(gd->decomps[to_add], new_gen, k);
    gd->contains[to_add] = true;
   }
  }
```

```
  k++;
  curr = calculate(g, curr, new_gen);
 }

 push_data_list(gd->generators, new_gen, k);

 int to_push;
 while(new_grp_elts->length > 0) {
  to_push = pop(new_grp_elts);
  push(gd->grp_elts, to_push);
 }
 free_list(new_grp_elts);

 int next_gen = 0;
 while (next_gen < gd->n && gd->contains[next_gen]) {
  next_gen++;
 }

 return next_gen;
}

int calculate_next_gen_and_rel_fast(group g, group_data gd, gbg_collector coll, int new_gen,
int neutral) {
 assert(g != NULL && gd != NULL);
 assert(new_gen >= 0);

 if (!gd->initialised) {
  init_empty_group_data(gd, neutral);
 }
 list new_grp_elts = create_empty_list();
 int curr = new_gen;
 int k = 1;
 int to_add, x;

 while (!gd->contains[curr]) {
  for (node_t* node = gd->grp_elts->head; node != NULL; node = node->next) {
   x = node->val;
   to_add = calculate(g, curr, x);

   if (!gd->contains[curr]) {
    push(new_grp_elts, curr);
    // push_data_list(gd->decomps[curr], new_gen, k);
    append_data_in_tabl_only(gd->decomps, gd->n, curr, new_gen, k, coll);
    gd->contains[curr] = true;
   }

   if (!gd->contains[to_add]) {
    push(new_grp_elts, to_add);
    // del_copy_from(&gd->decomps[to_add], gd->decomps[x]);
    append_data_and_link_in_tabl(gd->decomps, gd->n, to_add, x, new_gen, k, coll);
    gd->contains[to_add] = true;
   }
  }

  k++;
  curr = calculate(g, curr, new_gen);
 }

 push_data_list(gd->generators, new_gen, k);

 int to_push;
 while(new_grp_elts->length > 0) {
  to_push = pop(new_grp_elts);
  push(gd->grp_elts, to_push);
 }
 free_list(new_grp_elts);
```

```
 int next_gen = 0;
 while (next_gen < gd->n && gd->contains[next_gen]) {
  next_gen++;
 }

 return next_gen;
}


group_data calculate_group_data(group g) {
 int neutral = find_neutral(g);

 group_data gd = create_empty_group_data(g->n);
 init_empty_group_data(gd, neutral);

 int gen = 0;

 while (gen < g->n && gd->grp_elts->length < g->n) {
  gen = calculate_next_gen_and_rel(g, gd, gen, neutral);
 }

 return gd;
}

group_data calculate_group_data_fast(group g, gbg_collector trash) {
 int neutral = find_neutral(g);

 group_data gd = create_empty_group_data(g->n);
 init_empty_group_data(gd, neutral);

 int gen = 0;

 while (gen < g->n && gd->grp_elts->length < g->n) {
  gen = calculate_next_gen_and_rel_fast(g, gd, trash, gen, neutral);
 }

 return gd;
}

void check_group_data(group g, group_data gd, int neutral) {
 for (int i = 0; i < g->n; i++) {
  int res = neutral;
  for (node_data c = gd->decomps[i]->head; c != NULL; c=c->next) {
   int val = fast_exp(g, c->g, neutral, c->p);
   res = calculate(g, res, val);
  }
  if (i != res) {
   printf("[-] ERROR: %d != %d.\n", i, res);
  }
  assert(res == i);
 }
}

void print_arr(int* arr, int n) {
 for (int i = 0; i < n; i++) {
  printf("%d ", arr[i]);
 }
 printf("\n");
}


matrix get_trig_rel_table(group g, int neutral, group_data gd) {
 assert(gd != NULL && gd->generators != NULL);

 int t = gd->generators->length;
 matrix trig_rel = mat_alloc(t);
```

```
 int *col_of = malloc((size_t)g->n * sizeof(int));
 assert(col_of != NULL);
 for (int x = 0; x < g->n; x++) col_of[x] = -1;

 int col = 0;
 for (node_data gen = gd->generators->head; gen != NULL; gen = gen->next) {
  assert(gen->g >= 0 && gen->g < g->n);
  col_of[gen->g] = col++;
 }
 assert(col == t);

 int i = 0;
 for (node_data gen = gd->generators->head; gen != NULL; gen = gen->next) {
  int curr_elt = fast_exp(g, gen->g, neutral, gen->p);
  data_list decomp = gd->decomps[curr_elt];

  trig_rel->t[i][i] -= gen->p;

  for (node_data c = decomp->head; c != NULL; c = c->next) {
   assert(c->g >= 0 && c->g < g->n);
   int j = col_of[c->g];
   assert(j >= 0 && j < t);
   trig_rel->t[i][j] += c->p;
  }
  i++;
 }

 free(col_of);
 return trig_rel;
}

basis create_empty_basis(int n) {
 basis b = malloc(sizeof(basis_s));
 b->n = n;
 b->elts = calloc(n, sizeof(int));
 b->order = calloc(n, sizeof(int));
 return b;
}
void free_basis(basis b) {
 free(b->elts);
 free(b->order);
 free(b);
}

void check_basis(group g, basis b, int neutral) {
 assert(is_neutral(g, neutral));
 for (int i = 0; i < b->n; i++) {
  int val = fast_exp(g, b->elts[i], neutral, b->order[i]);
  assert(val == neutral);
 }
}

void show_basis(basis b) {
 printf("Order:\n");
 print_arr(b->order, b->n);
 printf("Elements:\n");
 print_arr(b->elts, b->n);
}

basis get_basis_from_data(group g, group_data gd, int neutral, matrix D, matrix V_inv) {
 int t = gd->generators->length;
 printf("Generators: ");
 show_values_data_list(gd->generators);
 printf("\n");
 basis b = create_empty_basis(t);
 for (int i = 0; i < t; i++) {
```

```
  b->elts[i] = neutral;
  int j = 0;
  for (node_data gen = gd->generators->head; gen != NULL; gen=gen->next) {
   int to_add = fast_exp(g, gen->g, neutral, ((V_inv->t[i][j]) % g->n) + g->n);
   b->elts[i] = calculate(g, b->elts[i], to_add);
   j++;
  }
  b->order[i] = D->t[i][i];
 }

 return b;
}

basis get_basis_from_rel(group g, group_data gd, int neutral) {
 int t = gd->generators->length;
 matrix R = get_trig_rel_table(g, neutral, gd);
 matrix U = identiy(t);
 matrix V = identiy(t);
 matrix D = mat_alloc(t);
 mat_snf_with_diagonal(R, U, V, D);
 // mat_print("U", U);
 // mat_print("V", V);
 // mat_print("D", D);

 matrix V_inv = mat_inverse(V);
 // mat_print("V inverse", V_inv);
 mat_free(R);
 mat_free(U);
 mat_free(V);

 basis b = get_basis_from_data(g, gd, neutral, D, V_inv);
 mat_free(D);
 mat_free(V_inv);

 return b;
}

basis get_basis(group g) {
 int neutral = find_neutral(g);
 printf("[*] En train de calculer les données du groupe.\n");
 group_data gd = calculate_group_data(g);
 basis b = get_basis_from_rel(g, gd, neutral);
 free_group_data(gd);
 printf("Base: ");
 print_arr(b->elts, b->n);

 printf("Ordres: ");
 print_arr(b->order, b->n);

 return b;
}

bool are_isomorphic(group g1, group g2) {
 int neutral1 = find_neutral(g1);
 group_data gd1 = calculate_group_data(g1);
 matrix R1 = get_trig_rel_table(g1, neutral1, gd1);
 free_group_data(gd1);

 matrix U1 = identity(R1->n);
 matrix V1 = identity(R1->n);
 matrix D1 = mat_alloc(R1->n);

 mat_snf_with_diagonal(R1, U1, V1, D1);

 mat_free(R1);
 mat_free(U1);
 mat_free(V1);
```

```
 int neutral2 = find_neutral(g2);
 group_data gd2 = calculate_group_data(g2);
 matrix R2 = get_trig_rel_table(g2, neutral2, gd2);
 free_group_data(gd2);

 matrix U2 = identity(R2->n);
 matrix V2 = identity(R2->n);
 matrix D2 = mat_alloc(R2->n);

 mat_snf_with_diagonal(R2, U2, V2, D2);

 mat_free(R2);
 mat_free(U2);
 mat_free(V2);

 bool res = same_abelian_group_from_snf(D1, D2);

 mat_free(D1);
 mat_free(D2);

 return res;
}

void iterate_decomps(int* decomps, int dim, int i, int* orders) {
 if (i >= dim) {
  return;
 } else {
  if ((decomps[i] + 1) % orders[i] == 0) {
   decomps[i] = 0;
   iterate_decomps(decomps, dim, i+1, orders);
  } else {
   decomps[i] += 1;
  }
 }
}


int* calculate_isomorphism(group g1, group g2) {
 int neutral1 = find_neutral(g1);
 group_data gd1 = calculate_group_data(g1);
 basis b1 = get_basis_from_rel(g1, gd1, neutral1);

 int neutral2 = find_neutral(g2);
 group_data gd2 = calculate_group_data(g2);
 basis b2 = get_basis_from_rel(g2, gd2, neutral2);

 int* iso = malloc(g1->n*sizeof(int));
 for (int i = 0; i < g1->n; i++) {
  iso[i] = -1;
 }
 int i = b1->n - 1;
 int j = b2->n - 1;
 int dim = 0;

 while (b1->elts[i] != neutral1 && i >= 0 && b2->elts[j] != neutral2 && j >= 0) {
  iso[b1->elts[i]] = b2->elts[j];
  i--;
  j--;
  dim++;
 }

 int* real_basis_g1 = calloc(dim, sizeof(int));
 int* real_basis_g2 = calloc(dim, sizeof(int));
 int* orders = calloc(dim, sizeof(int));

 for (int i = 0; i < dim; i++) {
```

```
  real_basis_g1[i] = b1->elts[b1->n-1 - i];
  real_basis_g2[i] = b2->elts[b2->n-1 - i];
  orders[i] = b1->order[b1->n-1 - i];
 }

 int* decomps = calloc(dim, sizeof(int));

 for (int k = 0; k < g1->n; k++) {
  int curr_elt_g1 = neutral1;
  int curr_elt_g2 = neutral2;
  for (int i = 0; i < dim; i++) {
   int vec1 = real_basis_g1[i];
   int vec2 = real_basis_g2[i];
   curr_elt_g1 = calculate(g1, curr_elt_g1, fast_exp(g1, vec1, neutral1, decomps[i]));
   curr_elt_g2 = calculate(g2, curr_elt_g2, fast_exp(g2, vec2, neutral2, decomps[i]));
  }
  iso[curr_elt_g1] = curr_elt_g2;

  iterate_decomps(decomps, dim, 0, orders);
 }

 free(decomps);
 free(real_basis_g1);
 free(real_basis_g2);
 free(orders);

 free_group_data(gd1);
 free_group_data(gd2);

 free_basis(b1);
 free_basis(b2);

 return iso;
}

bool check_isomorphism(group g1, group g2, int* iso) {
 assert(g1->n == g2->n);
 int n = g1->n;
 for (int x = 0; x < n; x++) {
  for (int y = 0; y < n; y++) {
   int e_g1 = iso[calculate(g1, x, y)];
   int e_g2 = calculate(g2, iso[x], iso[y]);
   bool check = (e_g1 == e_g2);
   if (!check) {
    printf("[-] For (%d, %d): %d != %d.\n", x, y, e_g1, e_g2);
    return false;
   }
  }
 }

 return true;
}
```

```c
#ifndef SMITH_NORMAL_FORM_H
#define SMITH_NORMAL_FORM_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

typedef long long ll;

typedef struct {
 int  n;
 ll **t;
} matrix_s;

typedef matrix_s *matrix;

matrix mat_alloc(int n);
void set_matrix(int n, matrix M, int m[][n]);
matrix from_given_matrix(int n, int m[][n]);
matrix identiy(int n);
matrix identity(int n);
void mat_free(matrix A);

void mat_identity(matrix A);
void mat_print(const char *name, matrix A);

void smith_normal_form(matrix M, matrix U, matrix V);

void mat_mul(matrix C, matrix A, matrix B);
matrix get_prod(matrix A, matrix B);
bool mat_equal(matrix A, matrix B);
bool same_abelian_group_from_snf(matrix D1, matrix D2);

int mat_inverse_bareiss(matrix M, matrix adj, ll *det_out);
matrix mat_inverse(matrix M);
matrix mat_snf_V_inv(matrix R);
void mat_snf_with_diagonal(matrix M, matrix U, matrix V, matrix D);
#endif
```

```c
#include "snf.h"

static ll extended_gcd(ll a, ll b, ll *x, ll *y) {
 if (b == 0) {
  *x = 1;
  *y = 0;
  return a;
 }
 ll x1, y1, g;

 g  = extended_gcd(b, a % b, &x1, &y1);
 *x = y1;
 *y = x1 - (a / b) * y1;
 return g;
}

matrix mat_alloc(int n) {
 if (n <= 0)
  return NULL;

 matrix A = (matrix)malloc(sizeof(matrix_s));
 if (A == NULL)
  return NULL;

 A->n = n;
 A->t = (ll **)calloc((size_t)n, sizeof(ll *));
 if (A->t == NULL) {
  free(A);
  return NULL;
 }

 int i = 0;
 while (i < n) {
  A->t[i] = (ll *)calloc((size_t)n, sizeof(ll));
  if (A->t[i] == NULL) {
   while (--i >= 0)
    free(A->t[i]);
   free(A->t);
   free(A);
   return NULL;
  }
  i++;
 }
 return A;
}

void set_matrix(int n, matrix M, int m[][n]) {
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   M->t[i][j] = m[i][j];
  }
 }
}

matrix from_given_matrix(int n, int m[][n]) {
 matrix M = mat_alloc(n);
 set_matrix(n, M, m);
 return M;
}

matrix identiy(int n) {
 matrix res = mat_alloc(n);
 for(int i = 0; i < n; i++) {
  res->t[i][i] = 1;
 }
 return res;
}
```

```
matrix identity(int n) {
 return identiy(n);
}

void mat_free(matrix A) {
 if (A == NULL)
  return;
 int i = 0;
 while (i < A->n) {
  free(A->t[i]);
  i++;
 }
 free(A->t);
 free(A);
}

void mat_identity(matrix A) {
 int i = 0;
 while (i < A->n) {
  int j = 0;
  while (j < A->n) {
   A->t[i][j] = (i == j) ? 1 : 0;
   j++;
  }
  i++;
 }
}

void mat_print(const char *name, matrix A) {
 int i, j;

 printf("%s (%dx%d):\n", name, A->n, A->n);
 i = 0;
 while (i < A->n) {
  printf("  [");
  j = 0;
  while (j < A->n) {
   printf(" %4lld", A->t[i][j]);
   j++;
  }
  printf(" ]\n");
  i++;
 }
}

static void swap_rows(matrix A, matrix U, int i, int j) {
 int k;

 if (i == j)
  return;
 k = 0;
 while (k < A->n) {
  ll t       = A->t[i][k];
  A->t[i][k] = A->t[j][k];
  A->t[j][k] = t;
  k++;
 }
 if (U) {
  k = 0;
  while (k < U->n) {
   ll t       = U->t[i][k];
   U->t[i][k] = U->t[j][k];
   U->t[j][k] = t;
   k++;
  }
 }
```

```
}

static void swap_cols(matrix A, matrix V, int i, int j) {
 int k;

 if (i == j)
  return;
 k = 0;
 while (k < A->n)
 {
  ll t        = A->t[k][i];
  A->t[k][i]  = A->t[k][j];
  A->t[k][j]  = t;
  k++;
 }
 if (V)
 {
  k = 0;
  while (k < V->n)
  {
   ll t        = V->t[k][i];
   V->t[k][i]  = V->t[k][j];
   V->t[k][j]  = t;
   k++;
  }
 }
}

static void row_combine(matrix A, matrix U, int r0, int r1, ll a, ll b, ll c, ll d) {
 int k = 0;
 while (k < A->n) {
  ll v0 = A->t[r0][k];
  ll v1 = A->t[r1][k];

  A->t[r0][k] = a * v0 + b * v1;
  A->t[r1][k] = c * v0 + d * v1;
  k++;
 }
 if (U) {
  k = 0;
  while (k < U->n) {
   ll v0 = U->t[r0][k];
   ll v1 = U->t[r1][k];

   U->t[r0][k] = a * v0 + b * v1;
   U->t[r1][k] = c * v0 + d * v1;
   k++;
  }
 }
}

static ll ll_abs_snf(ll x) {
 return (x < 0) ? -x : x;
}

static void add_row_multiple(matrix A, matrix U, int dst, int src, ll k) {
 if (k == 0 || dst == src)
  return;
 for (int j = 0; j < A->n; j++)
  A->t[dst][j] += k * A->t[src][j];
 if (U) {
  for (int j = 0; j < U->n; j++)
   U->t[dst][j] += k * U->t[src][j];
 }
}

static void add_col_multiple(matrix A, matrix V, int dst, int src, ll k) {
```

```
 if (k == 0 || dst == src)
  return;
 for (int i = 0; i < A->n; i++)
  A->t[i][dst] += k * A->t[i][src];
 if (V) {
  for (int i = 0; i < V->n; i++)
   V->t[i][dst] += k * V->t[i][src];
 }
}

static void negate_row(matrix A, matrix U, int row) {
 for (int j = 0; j < A->n; j++)
  A->t[row][j] = -A->t[row][j];
 if (U) {
  for (int j = 0; j < U->n; j++)
   U->t[row][j] = -U->t[row][j];
 }
}

static int has_offdiag_in_pivot_cross(matrix M, int p) {
 for (int i = p + 1; i < M->n; i++)
  if (M->t[i][p] != 0 || M->t[p][i] != 0)
   return 1;
 return 0;
}

static void reduce_pivot_cross(matrix M, matrix U, matrix V, int p) {
 int n = M->n;

 while (has_offdiag_in_pivot_cross(M, p)) {
  for (int i = p + 1; i < n; i++) {
   while (M->t[i][p] != 0) {
    if (ll_abs_snf(M->t[i][p]) < ll_abs_snf(M->t[p][p]))
     swap_rows(M, U, p, i);
    ll q = M->t[i][p] / M->t[p][p];
    if (q == 0)
     q = (M->t[i][p] > 0) == (M->t[p][p] > 0) ? 1 : -1;
    add_row_multiple(M, U, i, p, -q);
   }
  }

  for (int j = p + 1; j < n; j++) {
   while (M->t[p][j] != 0) {
    if (ll_abs_snf(M->t[p][j]) < ll_abs_snf(M->t[p][p]))
     swap_cols(M, V, p, j);
    ll q = M->t[p][j] / M->t[p][p];
    if (q == 0)
     q = (M->t[p][j] > 0) == (M->t[p][p] > 0) ? 1 : -1;
    add_col_multiple(M, V, j, p, -q);
   }
  }
 }
}

void smith_normal_form(matrix M, matrix U, matrix V) {
 int n = M->n;

 for (int p = 0; p < n; p++) {
  int found = 0;
  int bi = p;
  int bj = p;
  ll best = 0;

  for (int i = p; i < n; i++) {
   for (int j = p; j < n; j++) {
    ll av = ll_abs_snf(M->t[i][j]);
    if (av != 0 && (!found || av < best)) {
```

```
      found = 1;
      best = av;
      bi = i;
      bj = j;
     }
    }
   }

   if (!found)
    break;

   swap_rows(M, U, p, bi);
   swap_cols(M, V, p, bj);

   for (;;) {
    reduce_pivot_cross(M, U, V, p);

    ll piv = M->t[p][p];
    int bad_i = -1;
    for (int i = p + 1; i < n && bad_i < 0; i++) {
     for (int j = p + 1; j < n; j++) {
      if (M->t[i][j] % piv != 0) {
       bad_i = i;
       break;
      }
     }
    }

    if (bad_i < 0)
     break;
    add_row_multiple(M, U, p, bad_i, 1);
   }

   if (M->t[p][p] < 0)
    negate_row(M, U, p);
  }
}

void mat_mul(matrix C, matrix A, matrix B) {
 int n = A->n;
 int i, j, k;

 i = 0;
 while (i < n) {
  j = 0;
  while (j < n) {
   C->t[i][j] = 0;
   j++;
  }
  i++;
 }
 i = 0;
 while (i < n) {
  k = 0;
  while (k < n) {
   ll aik = A->t[i][k];
   if (aik != 0) {
    j = 0;
    while (j < n) {
     C->t[i][j] += aik * B->t[k][j];
     j++;
    }
   }
   k++;
  }
  i++;
 }
```

```
}

matrix get_prod(matrix A, matrix B) {
 matrix C = mat_alloc(A->n);
 mat_mul(C, A, B);
 return C;
}

bool mat_equal(matrix A, matrix B) {
 if (A->n != B->n) return false;
 int n = A->n;
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   if (A->t[i][j] != B->t[i][j]) {
    return false;
   }
  }
 }
 return true;
}

static int det_bareiss_array(const ll *src, int n, ll *det_out) {
 if (n <= 0 || src == NULL || det_out == NULL)
  return -1;
 if (n == 1) {
  *det_out = src[0];
  return 0;
 }

 ll *a = (ll *)malloc((size_t)n * (size_t)n * sizeof(ll));
 if (a == NULL)
  return -1;
 memcpy(a, src, (size_t)n * (size_t)n * sizeof(ll));

 ll prev = 1;
 int sign = 1;

 for (int k = 0; k < n - 1; k++) {
  int pivot = k;
  ll best = ll_abs_snf(a[(size_t)k * (size_t)n + (size_t)k]);
  for (int i = k + 1; i < n; i++) {
   ll av = ll_abs_snf(a[(size_t)i * (size_t)n + (size_t)k]);
   if (av > best) {
    best = av;
    pivot = i;
   }
  }

  if (best == 0) {
   *det_out = 0;
   free(a);
   return 0;
  }

  if (pivot != k) {
   for (int j = 0; j < n; j++) {
    ll tmp = a[(size_t)k * (size_t)n + (size_t)j];
    a[(size_t)k * (size_t)n + (size_t)j] =
     a[(size_t)pivot * (size_t)n + (size_t)j];
    a[(size_t)pivot * (size_t)n + (size_t)j] = tmp;
   }
   sign = -sign;
  }

  ll piv = a[(size_t)k * (size_t)n + (size_t)k];
  for (int i = k + 1; i < n; i++) {
   for (int j = k + 1; j < n; j++) {
```

```
    ll val = piv * a[(size_t)i * (size_t)n + (size_t)j]
     - a[(size_t)i * (size_t)n + (size_t)k]
      * a[(size_t)k * (size_t)n + (size_t)j];
    a[(size_t)i * (size_t)n + (size_t)j] = val / prev;
   }
  }
  prev = piv;

  for (int i = k + 1; i < n; i++)
   a[(size_t)i * (size_t)n + (size_t)k] = 0;
 }

 *det_out = (ll)sign * a[(size_t)(n - 1) * (size_t)n + (size_t)(n - 1)];
 free(a);
 return 0;
}

static int matrix_det_bareiss(matrix M, ll *det_out) {
 int n = M->n;
 ll *buf = (ll *)malloc((size_t)n * (size_t)n * sizeof(ll));
 if (buf == NULL)
  return -1;
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++)
   buf[(size_t)i * (size_t)n + (size_t)j] = M->t[i][j];
 }
 int rc = det_bareiss_array(buf, n, det_out);
 free(buf);
 return rc;
}

static int minor_det_bareiss(matrix M, int skip_row, int skip_col, ll *det_out) {
 int n = M->n;
 if (n == 1) {
  *det_out = 1;
  return 0;
 }

 int m = n - 1;
 ll *minor = (ll *)malloc((size_t)m * (size_t)m * sizeof(ll));
 if (minor == NULL)
  return -1;

 int rr = 0;
 for (int i = 0; i < n; i++) {
  if (i == skip_row)
   continue;
  int cc = 0;
  for (int j = 0; j < n; j++) {
   if (j == skip_col)
    continue;
   minor[(size_t)rr * (size_t)m + (size_t)cc] = M->t[i][j];
   cc++;
  }
  rr++;
 }

 int rc = det_bareiss_array(minor, m, det_out);
 free(minor);
 return rc;
}

int mat_inverse_bareiss(matrix M, matrix adj, ll *det_out) {
 if (M == NULL || adj == NULL || det_out == NULL || M->n <= 0 || adj->n != M->n)
  return -1;

 int n = M->n;
```

```
 if (matrix_det_bareiss(M, det_out) != 0)
  return -1;

 if (*det_out == 0) {
  for (int i = 0; i < n; i++)
   for (int j = 0; j < n; j++)
    adj->t[i][j] = 0;
  return -1;
 }

 if (n == 1) {
  adj->t[0][0] = 1;
  return 0;
 }

 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   ll d = 0;
   if (minor_det_bareiss(M, i, j, &d) != 0)
    return -1;
   if (((i + j) & 1) != 0)
    d = -d;
   adj->t[j][i] = d;
  }
 }
 return 0;
}

matrix mat_inverse(matrix M) {
 if (M == NULL || M->n <= 0)
  return NULL;

 int n = M->n;
 matrix W = mat_alloc(n);
 matrix Inv = mat_alloc(n);
 if (W == NULL || Inv == NULL) {
  mat_free(W);
  mat_free(Inv);
  return NULL;
 }

 for (int i = 0; i < n; i++)
  memcpy(W->t[i], M->t[i], (size_t)n * sizeof(ll));
 mat_identity(Inv);

 for (int k = 0; k < n; k++) {
  int pivot_row = -1;
  for (int i = k; i < n; i++)
  {
   if (W->t[i][k] != 0) {
    pivot_row = i;
    break;
   }
  }
  if (pivot_row == -1) {
   mat_free(W);
   mat_free(Inv);
   return NULL;
  }
  swap_rows(W, Inv, k, pivot_row);

  for (int i = k + 1; i < n; i++) {
   while (W->t[i][k] != 0) {
    ll pivot = W->t[k][k];
    ll val = W->t[i][k];
    ll x, y, g;
```

```
    ll sp = (pivot < 0) ? -1 : 1;
    ll sv = (val < 0) ? -1 : 1;
    ll ap = pivot * sp;
    ll av = val * sv;
    g = extended_gcd(ap, av, &x, &y);
    x *= sp;
    y *= sv;

    row_combine(W, Inv, k, i, x, y, -(val / g), pivot / g);
   }
  }

  if (W->t[k][k] != 1 && W->t[k][k] != -1) {
   mat_free(W);
   mat_free(Inv);
   return NULL;
  }

  if (W->t[k][k] == -1) {
   for (int j = 0; j < n; j++) {
    W->t[k][j] = -W->t[k][j];
    Inv->t[k][j] = -Inv->t[k][j];
   }
  }

  for (int i = 0; i < n; i++) {
   if (i == k || W->t[i][k] == 0)
    continue;
   ll factor = W->t[i][k];
   for (int j = 0; j < n; j++) {
    W->t[i][j] -= factor * W->t[k][j];
    Inv->t[i][j] -= factor * Inv->t[k][j];
   }
  }
 }

 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   ll expected = (i == j) ? 1 : 0;
   if (W->t[i][j] != expected) {
    mat_free(W);
    mat_free(Inv);
    return NULL;
   }
  }
 }

 mat_free(W);
 return Inv;
}

matrix mat_snf_V_inv(matrix R) {
    int n = R->n;
    matrix U = mat_alloc(n);
    matrix V = mat_alloc(n);
    matrix M = mat_alloc(n);

    int i = 0;
    while (i < n) {
        memcpy(M->t[i], R->t[i], (size_t)n * sizeof(ll));
        i++;
    }

    mat_identity(U);
    mat_identity(V);
    smith_normal_form(M, U, V);
```

```
    matrix V_inv = mat_inverse(V);

    mat_free(U);
    mat_free(V);
    mat_free(M);
    return V_inv;
}

void mat_snf_with_diagonal(matrix M, matrix U, matrix V, matrix D) {
    int n = M->n;
    int i = 0;

    while (i < n) {
        memcpy(D->t[i], M->t[i], (size_t)n * sizeof(ll));
        i++;
    }

    mat_identity(U);
    mat_identity(V);
    smith_normal_form(D, U, V);
}

bool same_abelian_group_from_snf(matrix D1, matrix D2) {
    int i = 0;
    int j = 0;

    while (i < D1->n || j < D2->n) {
        while (i < D1->n && abs(D1->t[i][i]) == 1) i++;
        while (j < D2->n && abs(D2->t[j][j]) == 1) j++;

        if (i >= D1->n || j >= D2->n) {
            return i >= D1->n && j >= D2->n;
        }

        if (abs(D1->t[i][i]) != abs(D2->t[j][j])) {
            return false;
        }

        i++;
        j++;
    }

    return true;
}
```

```c
#ifndef GEN_TBL
#define GEN_TBL

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <assert.h>
#include "../Linked_List/linked_list.h"

typedef struct data_s {
 int g;
 int p;
} data_t;
typedef data_t* data;

typedef struct node_data_s {
 int g;
 int p;
 struct node_data_s* next;
} node_data_t;

typedef node_data_t* node_data;

typedef struct data_list_s {
  node_data head;
  node_data tail;
  int length;
} data_list_t;
typedef data_list_t* data_list;

node_data new_data_node(int g, int p);
bool is_empty_data_list(data_list dlist);

data_list create_empty_data_list();
data_list create_example_data_list(int n);
data_list create_single_data_list(int n);
data_list deep_copy_data_list(data_list dlist);
void del_copy_from(data_list* d1, data_list d2);

void show_values_data_list(data_list dlist);
void push_data_list(data_list dlist, int g, int p);
void append_data_list(data_list dlist, int g, int p);
data pop_data_list(data_list dlist);

node_data free_get_next_node_data(node_data node);
void free_data_list(data_list dlist);

typedef struct group_data_s {
 int n;
 bool initialised;

 bool* contains;
 list grp_elts;

 int* order;
 data_list generators;

 data_list* decomps;
} group_data_t;
typedef group_data_t* group_data;

group_data create_empty_group_data(int n);
void init_empty_group_data(group_data gd, int neutral);
void free_group_data(group_data tr);

void show_num_elts(group_data tr);
void show_rel(group_data tr);
```

```
void show_contains(group_data tr);
void show_grp_elts(group_data tr);
void show_generators(group_data tr);

#endif
```

```c
#include "gen_table.h"

data create_data(int g, int p) {
 data d = malloc(sizeof(data_t));
 d->g = g;
 d->p = p;
 return d;
}

node_data new_data_node(int g, int p) {
 node_data node = malloc(sizeof(node_data_t));
 node->g = g;
 node->p = p;
 node->next = NULL;
 return node;
}

data_list create_empty_data_list() {
 data_list new_data_list = malloc(sizeof(data_list_t));
 new_data_list->head = NULL;
 new_data_list->tail = NULL;
 new_data_list->length = 0;
 return new_data_list;
}

data_list create_example_data_list(int n) {
 data_list example = create_empty_data_list();
 for (int i = 0; i < n; i++) {
  push_data_list(example, i, i);
 }
 return example;
}

data_list create_single_data_list(int g) {
 data_list single = create_empty_data_list();
 push_data_list(single, g, 1);
 return single;
}

data_list deep_copy_data_list(data_list dlist) {
 data_list copy = create_empty_data_list();
 for (node_data c=dlist->head; c != NULL; c=c->next) {
  append_data_list(copy, c->g, c->p);
 }

 return copy;
}

void del_copy_from(data_list* d1, data_list d2) {
 assert(d1 != NULL && d2 != NULL);
 free_data_list(*d1);
 *d1 = deep_copy_data_list(d2);
}

bool is_empty_data_list(data_list dlist) {
 assert(dlist != NULL);
 return (dlist->length == 0 && dlist->head == NULL);
}

void show_values_data_list(data_list dlist) {
 assert(dlist != NULL);
 for (node_data c = dlist->head; c != NULL; c=c->next) {
  printf("(%d,%d) -> ", c->g, c->p);
 }
 printf("END\n");
}
```

```
void push_data_list(data_list dlist, int g, int p) {
 // printf("Adding: (%d, %d)\n", g, p);

 node_data new_head = new_data_node(g, p);
 node_data old_head = dlist->head;
 new_head->next = old_head;
 dlist->head = new_head;
 dlist->length += 1;

 if (dlist->length == 1) {
  dlist->tail = dlist->head;
 }
}

void append_data_list(data_list dlist, int g, int p) {
 if (dlist->length <= 0) {
  push_data_list(dlist, g, p);
 } else {
  node_data new_tail = new_data_node(g, p);
  dlist->tail->next = new_tail;
  dlist->tail = new_tail;
  dlist->length += 1;
 }
}

data pop_data_list(data_list dlist) {
 assert(dlist != NULL && !is_empty_data_list(dlist));
 node_data old_head = dlist->head;
 node_data new_head = old_head->next;
 dlist->head = new_head;

 data res = create_data(old_head->g, old_head->p);
 free(old_head);
 dlist->length--;
 return res;
}

node_data free_get_next_node_data(node_data node) {
 assert(node != NULL);
 node_data next = node->next;
 free(node);
 return next;
}

void free_data_list(data_list dlist) {
 assert(dlist != NULL);
 node_data curr = dlist->head;
 while (curr != NULL) {
  curr = free_get_next_node_data(curr);
 }

 free(dlist);
}

group_data create_empty_group_data(int n) {
 group_data res = malloc(sizeof(group_data_t));
 res->n = n;
 res->initialised = false;


 res->contains = calloc(n, sizeof(bool));
 res->order = malloc(n*sizeof(int));
 for (int i = 0; i < n; i++) {
  res->order[i] = -1;
 }

 res->grp_elts = create_empty_list();
```

```
 res->generators = create_empty_data_list();

 res->decomps = malloc(n*sizeof(data_list));
 for (int i = 0; i < n; i++) {
  res->decomps[i] = create_empty_data_list();
 }

 return res;
}

void init_empty_group_data(group_data gd, int neutral) {
 assert(gd != NULL && neutral >= 0 && neutral < gd->n);
 assert(!gd->initialised);
 assert(gd->generators->length == 0 && gd->grp_elts->length == 0);
 push(gd->grp_elts, neutral);
 gd->contains[neutral] = true;
 gd->initialised = true;
}

void free_group_data(group_data tr) {
 assert(tr != NULL);
 for (int i = 0; i < tr->n; i++) {
  free_data_list(tr->decomps[i]);
 }
 free(tr->decomps);
 free_list(tr->grp_elts);
 free_data_list(tr->generators);
 free(tr->order);
 free(tr->contains);
 free(tr);
}

void show_num_elts(group_data tr) {
 int num_elts = 0;
 for (int i = 0; i < tr->n; i++) {
  if (tr->contains[i]) {
   num_elts++;
  }
 }
 printf("Nombre d'éléménts: %d\n", num_elts);
}

void show_rel(group_data tr) {
 printf("Décompositions:\n");
 assert(tr != NULL && tr->decomps != NULL);
 for (int i = 0; i < tr->n; i++) {
  if (tr->decomps[i]->length >= 1) {
   printf("%d: ", i);
   show_values_data_list(tr->decomps[i]);
  }
 }
}

void show_contains(group_data tr) {
 assert(tr != NULL && tr->contains != NULL);
 for (int i = 0; i < tr->n; i++) {
  printf("Has %d: ", i);
  if (tr->contains[i]) {
   printf("TRUE\n");
  } else {
   printf("FALSE\n");
  }
 }
}

void show_grp_elts(group_data tr) {
 assert(tr != NULL && tr->grp_elts != NULL);
```

```
 show_values(tr->grp_elts);
}

void show_generators(group_data tr) {
 assert(tr != NULL && tr->generators != NULL);
 printf("Générateur(s): ");
 show_values_data_list(tr->generators);
}
```

```c
#ifndef GARBAGE_COLLECTOR_H
#define GARBAGE_COLLECTOR_H

#include "../Gen_Table/gen_table.h"

typedef struct gbg_node_s {
 node_data* to_free;
 struct gbg_node_s* next;
} gbg_node_t;
typedef gbg_node_t* gbg_node;

typedef struct gbg_collector_s {
 gbg_node head;
 int size;
} gbg_collector_t;
typedef gbg_collector_t* gbg_collector;

gbg_node create_gbg_node(node_data* nd_addr);
void free_gbg_node(gbg_node head_gbg_node);

gbg_collector create_empty_gbg_collector(void);
void push_gbg_collector(node_data* nd_addr, gbg_collector collector);
void free_gbg_collector(gbg_collector collector);
void print_gbg_collector(gbg_collector collector);

void free_decomp_table(data_list* tabl, int n, gbg_collector coll);

// New functions to append in decomp table.
void append_data_list_from_ptr(data_list dlist, node_data* ptr);
void append_data_in_tabl_only(data_list* tabl, int n, int i, int g, int p, gbg_collector
coll);
void append_data_and_link_in_tabl(data_list* tabl, int n, int i, int j, int g, int p,
gbg_collector coll);

void free_gbc_gdata(group_data gd, gbg_collector gbc);


#endif
```

```c
#include "garbage_collector.h"

gbg_node create_gbg_node(node_data* nd_addr) {
 gbg_node res = malloc(sizeof(gbg_node_t));
 res->to_free = nd_addr;
 res->next = NULL;

 return res;
}

void free_gbg_node(gbg_node head_gbg_node) {
 if (head_gbg_node == NULL) return;
 gbg_node next = head_gbg_node->next;
 free(*(head_gbg_node->to_free));
 free(head_gbg_node->to_free);
 free(head_gbg_node);

 free_gbg_node(next);
}

gbg_collector create_empty_gbg_collector(void) {
 gbg_collector res = malloc(sizeof(gbg_collector_t));
 res->head = NULL;
 res->size = 0;
 return res;
}

void push_gbg_collector(node_data* nd_addr, gbg_collector collector) {
 gbg_node new_head = create_gbg_node(nd_addr);
 new_head->next = collector->head;
 collector->head = new_head;
 collector->size++;
}

void free_gbg_collector(gbg_collector collector) {
 free_gbg_node(collector->head);
 free(collector);
}

void print_gbg_collector(gbg_collector collector) {
 assert(collector != NULL);
 printf("To free: %d\n", collector->size);
}

void free_decomp_table(data_list* tabl, int n, gbg_collector coll) {
 free_gbg_collector(coll);
 for (int i = 0; i < n; i++) {
  free(tabl[i]);
 }
 free(tabl);
}

void append_data_list_from_ptr(data_list dlist, node_data* ptr) {
 assert(dlist != NULL);
 if (dlist->length == 0) {
  dlist->head = *ptr;
  dlist->tail = *ptr;
 } else {
  dlist->tail->next = *ptr;
  dlist->tail = *ptr;
 }
 dlist->length++;
}

void append_data_in_tabl_only(data_list* tabl, int n, int i, int g, int p, gbg_collector coll)
{
 assert(tabl != NULL && i >= 0 && i < n);
```

```
 // Allocate fresh memory in the heap.
 node_data* ptr = malloc(sizeof(node_data));
 *ptr = new_data_node(g, p);

 append_data_list_from_ptr(tabl[i], ptr);

 push_gbg_collector(ptr, coll);

 // printf("Added value at index %d: (%d, %d)\n", i, g, p);
 // printf("Address of new tail in tabl: %p\n", ptr);
}

void append_data_and_link_in_tabl(data_list* tabl, int n, int i, int j, int g, int p,
gbg_collector coll) {
 assert(tabl != NULL && i != j);
 assert(i >= 0 && i < n && j >= 0 && j < n);

 // Allocate fresh memory in the heap.
 node_data* ptr = malloc(sizeof(node_data));
 *ptr = new_data_node(g, p);

 append_data_list_from_ptr(tabl[i], ptr);

 tabl[i]->tail->next = tabl[j]->head;

 push_gbg_collector(ptr, coll);


 // printf("Added value at index %d: (%d, %d)\n", i, g, p);
 // printf("Address of new tail in tabl: %p\n", ptr);
}

void free_gbc_gdata(group_data gd, gbg_collector gbc) {
 // free_gbg_collector(gbc);
 // for (int i = 0; i < gd->n; i++) {
 //  free(gd->decomps[i]);
 // }
 // free(gd->decomps);
 free_decomp_table(gd->decomps, gd->n, gbc);
 free_list(gd->grp_elts);
 free_data_list(gd->generators);
 free(gd->order);
 free(gd->contains);
 free(gd);
}
```

```c
#ifndef LL
#define LL

#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <stdbool.h>

typedef struct node {
  int val;
  struct node* next;
} node_t;

typedef struct list {
  node_t* head;
  node_t* tail;
  int length;
} list_t;
typedef list_t* list;

node_t* new_node(int n);

list_t* create_empty_list();
list_t* create_example(int n);
list_t* create_single(int n);

list_t* deep_copy(list_t* l);

bool is_empty(list_t* linked);

void show_list(list_t* linked);
void show_values(list_t* linked);

void push(list_t* linked, int n);
void append(list_t* linked, int n);
void insert(list_t* linked, int index, int x);
int pop(list_t* linked);

node_t* free_get_next(node_t* node);
void free_list(list_t* linked);

#endif
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <stdbool.h>

#include "linked_list.h"

node_t* new_node(int n) {
    node_t* out = malloc(sizeof(node_t));
    out->next = NULL;
    out->val = n;
    return out;
}

list_t* create_empty_list() {
    list_t* linked = malloc(sizeof(list_t));
    linked->head = NULL;
    linked->tail = NULL;
    linked->length = 0;
    return linked;
}

list_t* create_example(int n) {
    list_t* linked = create_empty_list();
    for (int i = 0; i < n; i++) {
        append(linked, i);
    }
    return linked;
}

list_t* create_single(int n) {
    list_t* linked = create_empty_list();
    push(linked, n);
    return linked;
}

list_t* deep_copy(list_t* l) {
    list_t* copy = create_empty_list();
    node_t* to_append;
    for (node_t* c = l->head; c != NULL; c = c->next) {
        append(copy, c->val);
    }

    return copy;
}

bool is_empty(list_t* linked) {
    return (linked->length == 0);
}

void show_list(list_t* linked) {
    printf("TĂŞte: %d\n", linked->head->val);
    printf("Queue: %d\n", linked->tail->val);
    printf("Longueur: %d\n", linked->length);

    node_t* curr = linked->head;
    while (curr != NULL) {
        printf("%d -> ", curr->val);
        curr = curr->next;
    }
    printf("FIN\n");
}

void show_values(list_t* linked) {
    for (node_t* c = linked->head; c != NULL; c = c->next) {
        printf("%d -> ", c->val);
    }
```

```
    printf("NULL\n");
}

void push(list_t* linked, int n) {
    node_t* new_head = malloc(sizeof(node_t));
    new_head->val = n;

    node_t* old_head = linked->head;
    new_head->next = old_head;
    linked->head = new_head;
    linked->length += 1;

    if (linked->length == 1) {
        linked->tail = new_head;
    }
}

void append(list_t* linked, int n) {
    assert(linked != NULL);
    if (linked->length == 0) {
        node_t* new_tail = new_node(n);
        linked->tail = new_tail;
        linked->head = new_tail;
        linked->length++;
    } else {
        node_t* new_tail = malloc(sizeof(node_t));
        new_tail->val = n;
        new_tail->next = NULL;

        node_t* old_tail = linked->tail;
        old_tail->next = new_tail;
        linked->tail = new_tail;
        linked->length += 1;
    }
}

void insert(list_t* linked, int index, int x) {
    assert(index <= linked->length);
    if (index == 0) {
        push(linked, x);
    }
    if (index == linked->length) {
        append(linked, x);
    }
    else {
        node_t* new_node = malloc(sizeof(node_t));
        new_node->val = x;

        node_t* curr = linked->head;
        int i = 0;
        while (i < index-1) {
            curr = curr->next;
            i += 1;
        }
        new_node->next = curr->next;
        curr->next = new_node;

        linked->length += 1;
    }
}

int pop(list_t* linked) {
    assert(is_empty(linked) == false);
    node_t* new_head = linked->head->next;
    node_t* old_head = linked->head;
    int val = old_head->val;
    linked->head = new_head;
```

```
    free(old_head);
    linked->length--;
    return val;
}

node_t* free_get_next(node_t* node) {
    assert(node != NULL);
    node_t* next = node->next;
    free(node);
    return next;
}

void free_list(list_t* linked) {
    assert(linked != NULL);
    node_t* curr = linked->head;
    while (curr != NULL) {
        curr = free_get_next(curr);
    }
    free(linked);
}
```

```c
#include "group.h"

int *zero_array(int n) {
 int *res = malloc(n * sizeof(int));
 for (int i = 0; i < n; i++) {
  res[i] = 0;
 }
 return res;
}

void group_name(group_t *g, char *s) {
 g->name = s;
}

group group_empty(int n) {
 group new_empty_group = malloc(sizeof(group_t));
 new_empty_group->n = n;
 new_empty_group->table = malloc(n * sizeof(int *));
 for (int i = 0; i < n; i++) {
  new_empty_group->table[i] = zero_array(n);
 }
 return new_empty_group;
}

void group_free(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
 int n = g->n;
 for (int i = 0; i < n; i++) {
  free(g->table[i]);
 }
 free(g->table);
 free(g);
}


void group_print(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
  int n = g->n;
 printf("Groupe %s d'ordre %d\n", g->name, n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   printf("%d ", g->table[i][j]);
  }
  printf("\n");
 }
}

group from_matrix(int **m) {
 assert(m != NULL && "[-] NULL POINTER exception.\n");
 int n = sizeof(*m[0]) / sizeof(int);
 group res = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   res->table[i][j] = m[i][j];
  }
 }
 return res;
}

group copy_group(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
 int n = g->n;
 group copy = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   copy->table[i][j] = g->table[i][j];
  }
```

```
 }
 return copy;
}

int calculate(group g, int x, int y) {
 assert(y >= 0);
 if (x == -1) {
  return y;
 }
 return g->table[x][y];
}

bool is_neutral(group g, int x) {
 assert(g != NULL);
 assert(x >= 0 && x < g->n);
 int res;
 if (x < g->n - 1) {
  res = calculate(g, x, x+1);
  return (res == x+1);
 } else {
  res = calculate(g, x, 0);
  return (res == 0);
 }
}

int get_neutral(group  g) {
 assert(g != NULL);
 for (int i = 0; i < g->n; i++) {
  if (is_neutral(g, i)) {
   return i;
  }
 }
 printf("Erreur.\n");
 return -1;
}
```

```c
#include "group.h"

int *zero_array(int n) {
 int *res = malloc(n * sizeof(int));
 for (int i = 0; i < n; i++) {
  res[i] = 0;
 }
 return res;
}

void group_name(group_t *g, char *s) {
 g->name = s;
}

group group_empty(int n) {
 group new_empty_group = malloc(sizeof(group_t));
 new_empty_group->n = n;
 new_empty_group->table = malloc(n * sizeof(int *));
 for (int i = 0; i < n; i++) {
  new_empty_group->table[i] = zero_array(n);
 }
 return new_empty_group;
}

void group_free(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
 int n = g->n;
 for (int i = 0; i < n; i++) {
  free(g->table[i]);
 }
 free(g->table);
 free(g);
}


void group_print(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
  int n = g->n;
 printf("Groupe %s d'ordre %d\n", g->name, n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   printf("%d ", g->table[i][j]);
  }
  printf("\n");
 }
}

group from_matrix(int **m) {
 assert(m != NULL && "[-] NULL POINTER exception.\n");
 int n = sizeof(*m[0]) / sizeof(int);
 group res = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   res->table[i][j] = m[i][j];
  }
 }
 return res;
}

group copy_group(group g) {
 assert(g != NULL && "[-] NULL POINTER exception.\n");
 int n = g->n;
 group copy = group_empty(n);
 for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
   copy->table[i][j] = g->table[i][j];
  }
```

```
 }
 return copy;
}

int calculate(group g, int x, int y) {
 assert(y >= 0);
 if (x == -1) {
  return y;
 }
 return g->table[x][y];
}

bool is_neutral(group g, int x) {
 assert(g != NULL);
 assert(x >= 0 && x < g->n);
 int res;
 if (x < g->n - 1) {
  res = calculate(g, x, x+1);
  return (res == x+1);
 } else {
  res = calculate(g, x, 0);
  return (res == 0);
 }
}

int get_neutral(group  g) {
 assert(g != NULL);
 for (int i = 0; i < g->n; i++) {
  if (is_neutral(g, i)) {
   return i;
  }
 }
 printf("Erreur.\n");
 return -1;
}
```